

PF-3312 · PROYECTO 2 · BENCHMARKING DE SERVICIOS DE IA

AUTOR

Ney Fred Jiménez Campos

CARNÉ

B03230

CURSO

PF-3312 Laboratorio de
Agentes Virtuales Inteligentes

PROFESOR

Dr. Alexander Barquero

PROGRAMA

Posgrado en Computación e
Informática, UCR — I Ciclo
2026

Resumen ejecutivo

Se evaluaron **quince servicios** (cinco de lenguaje, cinco de voz a texto y cinco de texto a voz) en las seis dimensiones del enunciado: latencia, precisión, costo, privacidad, customización e integración. Cada uno se probó **cinco veces por input** (25 llamadas en LLM/TTS y 70 en STT con 14 audios), con intervalos de confianza al 95 %, detección de valores atípicos y una medición del **pipeline completo** (STT → LLM → TTS, 100 turnos; **100 exitosos**).

En LLM, **Groq** fue el más rápido y barato (695 ms, \$0,0002/llamada); sus intervalos no se cruzan con el siguiente competidor. **OpenAI GPT-4o** equilibra velocidad y calidad de respuesta. En voz, **Deepgram** (380 ms) y **Google TTS Neural2** (300 ms) fueron los más rápidos, lo que permite turnos en nube por debajo de 2 s (demo rápida ≈ **1,8 s** E2E). El **stack local** (Ollama + faster-whisper + Piper) no cobra API ni envía datos a internet, pero el turno E2E promedió **~62 s** en el hardware usado, así que no sirve para tiempo real. Medir el pipeline completo difirió hasta ~33 % frente a sumar cada parte por separado (Sección 3.0.8).

Depende del escenario: no hay un proveedor mejor en todo. Las recomendaciones de la sección 5 proponen combinaciones según el caso: demo en vivo, calidad alta, privacidad offline o cliente Unity/Azure.

Las corridas las corrí en mi laptop del curso (Quadro P2000, 32 GB RAM). El cuello de botella offline fue faster-whisper (~40 s por clip), no Piper ni Ollama solos.

1. Introducción y Objetivos

El proyecto mide **quince servicios** (cinco LLM, cinco STT, cinco TTS) que forman un agente por voz típico: micrófono, transcripción, modelo de lenguaje y síntesis. La lista mezcla APIs en nube, opciones baratas o

rápidas y tres componentes locales (Ollama, faster-whisper, Piper) para comparar también el despliegue offline.

El trabajo responde al enunciado en **seis dimensiones** (latencia, precisión, costo, privacidad, customización e integración), con corridas repetidas, intervalos al 95 % y una prueba del pipeline completo (STT → LLM → TTS). Al cierre se proponen stacks según el escenario: demo en vivo, calidad alta, privacidad o cliente Unity/Azure.

Objetivos.

- Registrar métricas reproducibles y dejar trazabilidad en CSV y `run_manifest.json`.
- Contrastar nube vs. local con los mismos inputs (Apéndice B).
- Sugerir combinaciones concretas para los cuatro escenarios del informe.

2. Metodología de Pruebas

2.1 Entorno

- **Hardware:** laptop Windows, NVIDIA Quadro P2000 (4 GB VRAM), 32 GB RAM, Intel i9-8950HK. ↓ **238,9 Mbps** / ↑ **65,3 Mbps** (asimétrica), ping 61 ms al servidor Managua (NI). El plan contratado con el ISP es ~750 Mbps; la medición de pared refleja las condiciones reales del día del benchmark, no la velocidad teórica del contrato.
- **Software:** Python 3.13, entorno virtual, repositorio [pf3312_lab_2](#).
- **Trazabilidad:** los resultados quedan en los CSV del repositorio con identificador de corrida por fila y un manifiesto de ejecución.

2.2 Inputs controlados

- **LLM:** cinco prompts en español (`common/prompts.py`); texto completo en **Apéndice B**.
- **STT:** catorce audios en siete fuentes: cuatro sintéticos Piper (`a1` , `a2` , `a4` , `a5`), tres FLEURS `es_419` (`g1` , `g2` , `g4`), dos Common Voice ruidosos (`c1` , `c3`), tres largos (`l1` concat FLEURS, `l2` Piper, `l4` ruido sintético) y dos América central (`r1` , `r2`); ver catálogo en `data/reference_transcriptions.json` y **Apéndice B**.
- **TTS:** cinco textos (`t1` - `t5` en `common/audio_samples.py`); ver **Apéndice B**.

2.3 Protocolo

- Cinco ejecuciones por input por servicio (`python -m run_all 5`).
- Latencia con `time.perf_counter` ; **TTFT** en LLM con streaming.
- **Costo** estimado según tarifas en cada script; locales documentados con VRAM/RAM.

- **STT:** WER (`jiwer`) frente a transcripción de referencia; desglose Piper vs FLEURS.
- **TTS:** MOS de inteligibilidad (WER round-trip STT→TTS) y naturalidad (escucha manual de `t2_oracion_media` por proveedor; ver `data/tts_mos_scores.json`).
- **LLM (dimensión 2):** revisión manual de las respuestas de la corrida LLM para coherencia e instrucciones complejas (`data/llm_quality_notes.json`).
- **Dimensiones 4-6:** rúbrica 1-5 aplicada servicio por servicio (Metodología cualitativa, Sección 2.6).

2.4 Herramientas

- SDKs cloud: OpenAI, Anthropic (`claude-sonnet-4-6`), Google Gemini/Cloud TTS, Groq, Deepgram, AssemblyAI, Azure Speech, ElevenLabs.
- Locales: Ollama (Llama 3.2 3B), faster-whisper (large-v3), Piper TTS.
- Análisis: `python scripts/run_analysis.py` (tablas, gráficos y matrices por dimensión).

2.5 Recursos locales (dimensión 3, escalabilidad)

- **Plataforma:** Windows-11-10.0.26200-SP0
- **CPU lógicos:** 12
- **RAM total:** 31.73 GB (disponible al capturar: 9.0 GB)

GPU	VRAM total (MB)	VRAM usada (MB)	Driver
Quadro P2000	4096	865	573.96

Energía (estimada): TDP de referencia de la GPU ~75 W (Quadro P2000). En inferencia local sostenida (faster-whisper/Ollama) el consumo del sistema se acerca a carga GPU+CPU; no se midió con wattímetro en esta corrida. Como referencia orientativa, una hora de inferencia local continua podría consumir del orden de 0,1-0,2 kWh (GPU+CPU+RAM), frente a un costo marginal de API en nube que escala linealmente con turnos (tabla siguiente). La comparación justa entre local y cloud requiere amortizar hardware, electricidad y tiempo de operación frente a Costo por turno conversacional (Sección 3.0.6).

2.6 Metodología cualitativa (dimensiones 4-6)

Las dimensiones de **privacidad**, **customización** e **integración** se puntúan del 1 al 5 con una rúbrica propia (escala documentada en Sección 2.6), contrastando políticas públicas y documentación de cada API (consulta mayo 2026). El resumen está en Sección 3.4; el detalle en Sección 3.0.9.

- **LLM (coherencia e instrucciones):** revisión manual de las respuestas de la corrida LLM, registrada en `data/llm_quality_notes.json` y contrastada con la tasa de JSON válido en p3 (Sección 3.0.4).
- **TTS (MOS):** inteligibilidad vía WER round-trip y naturalidad por escucha de `t2_oracion_media` (`data/tts_mos_scores.json` , Sección 3.0.5).

Tarifas en unidades estándar (dimensión 3)

Valores de referencia desde `common/rates.py` ; verificar en sitios oficiales antes de producción. El costo por llamada en CSV deriva de tokens, duración de audio o caracteres sintetizados en cada input.

LLM (USD por millón de tokens)

proveedor	modelo	usd_input_M_tokens	usd_output_M_tokens	fuelle
openai	gpt-4o	2.5	10	common/rates.py
anthropic	claude-sonnet-4-6	3	15	common/rates.py
google	gemini-2.5-flash	0.3	2.5	common/rates.py
groq	llama-3.3-70b-versatile	0.59	0.79	common/rates.py
ollama	llama3.2:3b	0	0	local (costo marginal API \$0)

STT (USD por minuto de audio transcrito)

proveedor	modelo	usd_minuto
deepgram	nova-3	0.0043
assemblyai	universal-3-pro	0.0036
openai	whisper-1	0.006
azure	speech-recognition-standard	0.0167
faster-whisper	large-v3	0

TTS (USD por millón de caracteres sintetizados)

proveedor	modelo	usd_M_caracteres
google	es-ES-Neural2-A	16
azure	es-CR-MariaNeural	16
elevenlabs	eleven_multilingual_v2	300
openai	gpt-4o-mini-tts	600
piper	es_ES-davefx-medium	0

Locales (Ollama, faster-whisper, Piper): costo API \$0; el costo real es infraestructura (VRAM/RAM, energía), detallada en Recursos locales (Sección 2.5).

Groq y Gemini concentran los menores costos por token en LLM; ElevenLabs y OpenAI TTS encabezan el costo por carácter. Azure STT es el cloud más caro por minuto en esta tabla, pero ofrece locale es-CR para clientes.NET/Unity.

Proyección de costo por volumen

Estimación lineal a partir del costo promedio por turno de la última corrida. No incluye descuentos por volumen ni costo de infraestructura local.

escenario	turnos_dia	usd_dia	usd_mes_30d
Demo rápida nube	100	0.42	12.68
Demo rápida nube	1000	4.23	126.81
Demo rápida nube	10000	42.27	1268.06
Calidad premium	100	5.42	162.67
Calidad premium	1000	54.22	1626.69
Calidad premium	10000	542.23	16266.9
Privacidad offline	100	0	0
Privacidad offline	1000	0	0
Privacidad offline	10000	0	0
Stack Azure	100	0.96	28.94
Stack Azure	1000	9.65	289.38
Stack Azure	10000	96.46	2893.85

A 30 días: Demo rápida nube va de ~\$13/mes (100 turnos/día) a ~\$1,268/mes (10 000/día). Calidad premium multiplica el costo unas quince veces. Stack Azure queda en el medio. Privacidad offline muestra \$0 de API (sin contar luz ni hardware; ver Sección 2.5).

3. Resultados

3.0 Resumen transversal

Abreviaturas en tablas y matrices: Lat. = latencia media (ms); TTFT = tiempo al primer token (solo LLM); WER = word error rate (solo STT); USD = costo estimado por llamada; Priv. / Cust. / Integr. = puntuación 1-5 del catálogo cualitativo; σ = desviación estándar; p95 = percentil 95; IC95% = intervalo de confianza al 95 %; N = llamadas agregadas.

Mapa de dimensiones y fuentes de datos

Dimensión	Fuente en el proyecto
1 Latencia	<code>results/*_results.csv</code> (automático)
2 Precisión	WER en STT (automático); LLM/TTS cualitativo
3 Costo	CSV (estimado por rates en cada script)
4 Privacidad	Catálogo cualitativo 1-5 (revisar políticas)
5 Customización	catálogo 1-5
6 Integración	catálogo 1-5

Cobertura del benchmark (15 servicios)

- **LLM:** 5 filas en matriz; CSV=sí
- **STT:** 5 filas en matriz; CSV=sí
- **TTS:** 5 filas en matriz; CSV=sí

Mejores por dimensión (solo filas con datos)

- Menor latencia: **tts/google** (300.1 ms)
- Menor WER (STT): **stt/assemblyai** (0.0471)
- Menor costo/llamada: **llm/ollama** (\$0.0)
- Mayor Privacidad (catálogo): **llm/ollama** (5/5)
- Mayor Customización (catálogo): **llm/anthropic** (5/5)
- Mayor Integración (catálogo): **llm/anthropic** (5/5)

Las subsecciones 3.0.1-3.0.9 reúnen tablas y gráficos de todas las capas. El detalle por LLM, STT y TTS está en las secciones 3.1-3.3.

3.0.1 Matriz maestra 15×6

Los 15 servicios en las seis dimensiones del enunciado. WER solo en STT; TTFT solo en LLM; Priv./Cust./Integr. van de 1 a 5. Gráficos en [docs/graficos_generados/](#).

Servicio	Lat. (ms)	TTFT	WER	USD	Priv.	Cust.	Integr.	Precisión
llm/anthropic	6916.0000	1346.4000	nan	0.0057	4	5	5	Coherencia (cual.)
llm/google	5928.3000	4832.6000	nan	0.0008	3	4	4	Coherencia (cual.)
llm/groq	695.2000	227.0000	nan	0.0002	4	3	4	Coherencia (cual.)
llm/ollama	36642.5000	1740.7000	nan	0.0000	5	4	3	Coherencia (cual.)
llm/openai	1780.9000	584.0000	nan	0.0020	3	5	5	Coherencia (cual.)
stt/assemblyai	4218.1000	nan	0.0471	0.0011	3	4	4	WER=0.0471
stt/azure	6891.9000	nan	0.0709	0.0049	3	4	4	WER=0.0709
stt/deepgram	428.5000	nan	0.0659	0.0013	3	4	4	WER=0.0659
stt/faster-whisper	37481.8000	nan	0.0499	0.0000	5	3	3	WER=0.0499
stt/openai	2127.1000	nan	0.0655	0.0018	3	3	5	WER=0.0655
tts/azure	848.9000	nan	nan	0.0027	3	4	4	MOS 1-5 (escucha manual)
tts/elevenlabs	2147.5000	nan	nan	0.0512	3	5	4	MOS 1-5 (escucha manual)
tts/google	300.1000	nan	nan	0.0027	3	4	4	MOS 1-5 (escucha manual)
tts/openai	3368.6000	nan	nan	0.1024	3	4	5	MOS 1-5 (escucha manual)
tts/piper	3025.8000	nan	nan	0.0000	5	2	2	MOS 1-5 (escucha manual)

Con los datos de la matriz, Groq y Google TTS lideran en velocidad en nube; Ollama, faster-whisper y Piper no pagan API pero en este laptop fueron lentos; OpenAI y Anthropic suman mejor en custom e integración del lado LLM.

3.0.2 Matrices 5×6 por categoría

Desglose de la matriz maestra por capa del pipeline (LLM, STT, TTS).

LLM

Prov.	Lat. (ms)	TTFT	WER	USD	Priv.	Cust.	Integr.	Precisión
anthropic	6916.0000	1346.4000	nan	0.0057	4	5	5	Coherencia (cual.)
google	5928.3000	4832.6000	nan	0.0008	3	4	4	Coherencia (cual.)
groq	695.2000	227.0000	nan	0.0002	4	3	4	Coherencia (cual.)
ollama	36642.5000	1740.7000	nan	0.0000	5	4	3	Coherencia (cual.)
openai	1780.9000	584.0000	nan	0.0020	3	5	5	Coherencia (cual.)

STT

Prov.	Lat. (ms)	TTFT	WER	USD	Priv.	Cust.	Integr.	Precisión
assemblyai	4218.1000	nan	0.0471	0.0011	3	4	4	WER=0.0471
azure	6891.9000	nan	0.0709	0.0049	3	4	4	WER=0.0709
deepgram	428.5000	nan	0.0659	0.0013	3	4	4	WER=0.0659
faster-whisper	37481.8000	nan	0.0499	0.0000	5	3	3	WER=0.0499
openai	2127.1000	nan	0.0655	0.0018	3	3	5	WER=0.0655

TTS

Prov.	Lat. (ms)	TTFT	WER	USD	Priv.	Cust.	Integr.	Precisión
azure	848.9000	nan	nan	0.0027	3	4	4	MOS 1-5 (escucha manual)
elevenlabs	2147.5000	nan	nan	0.0512	3	5	4	MOS 1-5 (escucha manual)
google	300.1000	nan	nan	0.0027	3	4	4	MOS 1-5 (escucha manual)
openai	3368.6000	nan	nan	0.1024	3	4	5	MOS 1-5 (escucha manual)
piper	3025.8000	nan	nan	0.0000	5	2	2	MOS 1-5 (escucha manual)

Desglose de la matriz maestra por capa.

LLM. Groq ganó en tiempo y precio; Ollama local no cobra pero en el Quadro P2000 no da para tiempo real. OpenAI y Anthropic van 5/5 en custom e integración del catálogo.

STT. Deepgram ~380 ms; AssemblyAI el WER más bajo. faster-whisper acerca el error sin API, pero ~39 s por clip en este equipo.

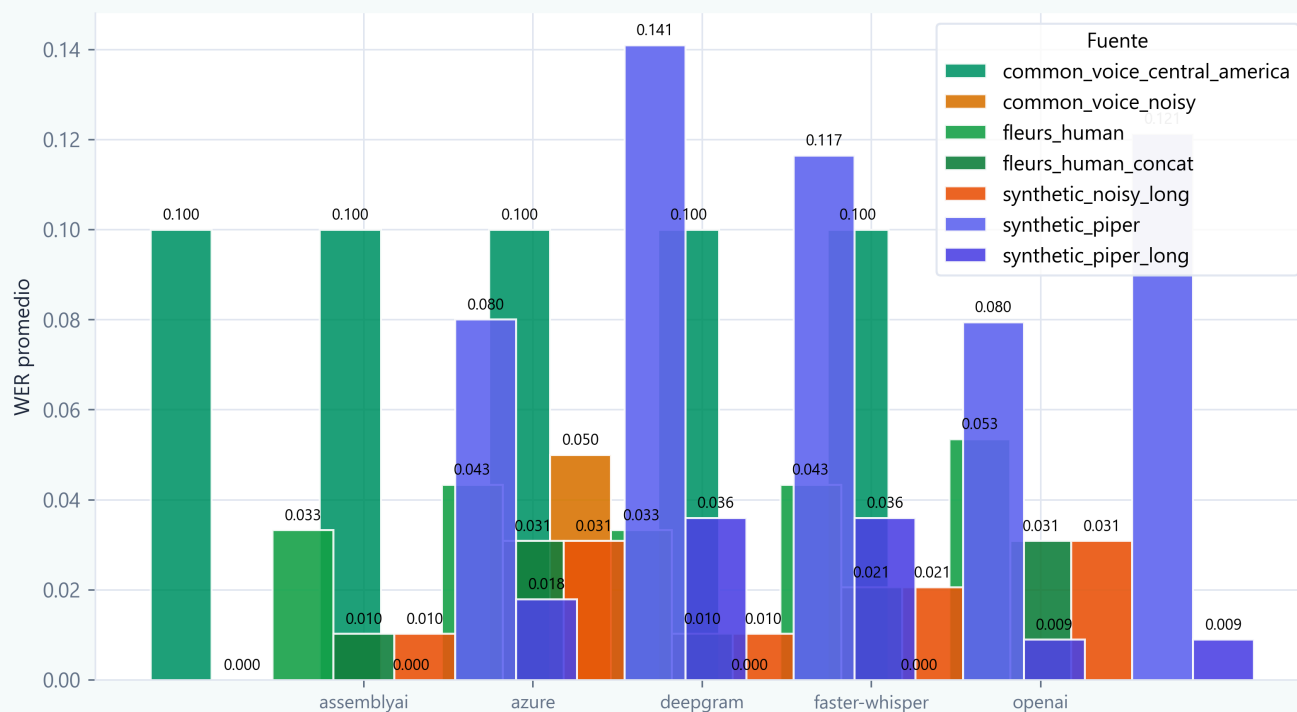
TTS. Google Neural2 barato y rápido; ElevenLabs sonó mejor en la escucha pero cuesta más. Piper sirve offline con timbre más robótico.

3.0.3 STT: WER por fuente de audio

provider	audio_source	wer_prom	llamadas
assemblyai	common_voice_central_america	0.1	10
assemblyai	common_voice_noisy	0	10
assemblyai	fleurs_human	0.0333	15

provider	audio_source	wer_prom	llamadas
assemblyai	fleurs_human_concat	0.0103	5
assemblyai	synthetic_noisy_long	0.0103	5
assemblyai	synthetic_piper	0.0801	20
assemblyai	synthetic_piper_long	0.018	5
azure	common_voice_central_america	0.1	10
azure	common_voice_noisy	0	10
azure	fleurs_human	0.0434	15
azure	fleurs_human_concat	0.0309	5
azure	synthetic_noisy_long	0.0309	5
azure	synthetic_piper	0.141	20
azure	synthetic_piper_long	0.036	5
deepgram	common_voice_central_america	0.1	10
deepgram	common_voice_noisy	0.05	10
deepgram	fleurs_human	0.0333	15
deepgram	fleurs_human_concat	0.0103	5
deepgram	synthetic_noisy_long	0.0103	5
deepgram	synthetic_piper	0.1165	20
deepgram	synthetic_piper_long	0.036	5
faster-whisper	common_voice_central_america	0.1	10
faster-whisper	common_voice_noisy	0	10
faster-whisper	fleurs_human	0.0434	15
faster-whisper	fleurs_human_concat	0.0206	5
faster-whisper	synthetic_noisy_long	0.0206	5
faster-whisper	synthetic_piper	0.0795	20
faster-whisper	synthetic_piper_long	0.009	5
openai	common_voice_central_america	0.1	10
openai	common_voice_noisy	0	10
openai	fleurs_human	0.0535	15
openai	fleurs_human_concat	0.0309	5
openai	synthetic_noisy_long	0.0309	5
openai	synthetic_piper	0.1213	20
openai	synthetic_piper_long	0.009	5

STT: WER por fuente de audio (Piper vs FLEURS)



WER por fuente de audio (STT)

Figura: WER promedio por proveedor y tipo de audio (Piper sintético vs FLEURS humano vs Common Voice). Barras más bajas = mejor transcripción.

El WER promedio por tipo de audio fue: Common Voice con ruido de fondo audible (SNR bajo) ~0.01; FLEURS concatenado largo (~30-40 s) ~0.02; ruido sintético sobre turno largo ~0.02; Piper monólogo largo ~0.02; voz humana FLEURS (limpio) ~0.04; CV América central (proxy acento CR) ~0.10; audio sintético Piper ~0.11. Con audio sintético (Piper) el error suele ser unas 3 veces mayor que con voz humana (FLEURS). Los clips con ruido (Common Voice *noisy*) suelen fallar más que FLEURS limpio. Las tablas generales de STT mezclan todas las fuentes; antes de comparar proveedores, revisar este desglose.

3.0.4 Calidad LLM

Dimensión 2 (precisión LLM): revisión manual de coherencia en p1/p2/p4/p5 y tasa de JSON válido en p3_json_estricto (escala 1-5 en las notas).

Prompt p3_json_estricto: cumplimiento de esquema JSON

provider	llamadas	json_valido	tasa_json
anthropic	5	5	1
google	5	3	0.6
groq	5	5	1
ollama	5	4	0.8

provider	llamadas	json_valido	tasa_json
openai	5	5	1

Coherencia global en p1/p2/p4/p5: `data/llm_quality_notes.json`.

Evaluador: Ney Fred Jiménez Campos (B03230) · Revisión manual de las respuestas de la corrida LLM: coherencia en p1/p2/p4/p5 y cumplimiento de formato en p3_json_estricto.

Puntuación cualitativa (escala 1–5)

provider	coherencia_1_5	instrucciones_1_5
openai	5	4
anthropic	5	3
google	4	4
groq	4	3
ollama	3	3

En p3_json_estricto, anthropic, groq, openai cumplieron el JSON en todas las repeticiones; con menor cumplimiento: google (60%), ollama (80%); coherencia manual p1/p2/p4/p5: openai 5/5, anthropic 5/5, google 4/5, groq 4/5, ollama 3/5. Para JSON estricto (p3), los modelos grandes en nube fallaron menos; detalle por prompt en el Apéndice B.

3.0.5 MOS TTS

Evaluador: Ney Fred Jiménez Campos (B03230) · Inteligibilidad vía WER (STT sobre audio sintetizado). Naturalidad: escucha de t2_oracion_medio por proveedor, escala 1-5.

provider	mos_inteligibilidad	mos_naturalidad	wer_promedio
azure	4	4	0.5999
elevenlabs	4	5	0.5492
google	3	4	0.5942
openai	4	4	0.5464
piper	3	3	0.5747

Al escuchar `t2_oracion_medio`, ElevenLabs se llevó el 5/5 en naturalidad; Google iba claro y rápido; Piper se nota sintético pero no paga API. La columna `wer_promedio` promedia los cinco textos; el round-trip solo dice si el audio se entiende al volver a transcribirlo, no si suena agradable.

3.0.6 Costo por turno conversacional

Promedio de costo/latencia por llamada en la última corrida filtrada.

escenario	llm	stt	tts	costo_usd_turno	latencia_ms_turno
Demo rápida nube	groq	deepgram	google	0.004227	1423.8
Calidad premium	openai	assemblyai	elevenlabs	0.054223	8146.5

escenario	llm	stt	tts	costo_usd_turno	latencia_ms_turno
Privacidad offline	ollama	faster-whisper	pipec	0	77150.1
Stack Azure	openai	azure	azure	0.009646	9521.7

En costo, Calidad premium (~\$0,053/turno) costó unas quince veces más que Demo rápida nube (~\$0,004); Privacidad offline no paga API pero la suma de latencias ronda 45 s por turno. Son sumas de componentes por separado; el pipeline E2E medido (Sección 3.0.8) difirió hasta $\pm 28\%$ en esta corrida.

3.0.7 Rigor empírico: intervalos de confianza y comparaciones

Cada proveedor acumula $n = \text{inputs} \times 5 \text{ runs}$ (LLM: 25 = 5×5; TTS: 25 = 5×5; STT: 70 = 14×5). Los IC95% usan la distribución t de Student sobre las llamadas exitosas de la corrida filtrada. Si los intervalos de dos proveedores no se cruzan, la diferencia es visible con el tamaño de muestra usado; no es un test estadístico formal.

Latencia total (ms) (LLM)

Proveedor	N	Media	σ	CV%	IC95% inf.	IC95% sup.
groq	25	695.24	456.04	65.6	506.88	883.6
openai	25	1780.93	864.34	48.5	1423.92	2137.93
google	25	5928.33	3220.95	54.3	4597.95	7258.71
anthropic	25	6916.05	4591.55	66.4	5019.56	8812.54
ollama	25	36642.5	33517.6	91.5	22798.4	50486.6

TTFT (ms) (LLM)

Proveedor	N	Media	σ	CV%	IC95% inf.	IC95% sup.
groq	25	226.99	49.3	21.7	206.62	247.35
openai	25	584.04	440.43	75.4	402.13	765.96
anthropic	25	1346.41	1287.52	95.6	814.61	1878.21
ollama	25	1740.66	1187.34	68.2	1250.24	2231.08
google	25	4832.61	2334.35	48.3	3868.44	5796.79

Latencia (ms) (STT)

Proveedor	N	Media	σ	CV%	IC95% inf.	IC95% sup.
deepgram	70	428.49	306.89	71.6	355.24	501.74
openai	70	2127.13	1084.11	51	1868.36	2385.89
assemblyai	70	4218.13	841.57	20	4017.25	4419
azure	70	6891.86	5764.4	83.6	5515.98	8267.75
faster-whisper	70	37481.8	20692.1	55.2	32542.9	42420.7

WER (STT)

Proveedor	N	Media	σ	CV%	IC95% inf.	IC95% sup.
assemblyai	70	0.05	0.08	161.6	0.03	0.07
faster-whisper	70	0.05	0.09	172.2	0.03	0.07
azure	70	0.07	0.09	131.5	0.05	0.09
deepgram	70	0.07	0.08	127.5	0.05	0.09
openai	70	0.07	0.09	143.2	0.04	0.09

Latencia (ms) (TTS)

Proveedor	N	Media	σ	CV%	IC95% inf.	IC95% sup.
google	25	300.05	181	60.3	225.29	374.81
azure	25	848.87	630.78	74.3	588.33	1109.4
elevenlabs	25	2147.46	1245.4	58	1633.06	2661.86
piiper	25	3025.79	2032.3	67.2	2186.37	3865.21
openai	25	3368.56	1483.2	44	2755.94	3981.18

Groq fue el LLM más rápido; Deepgram el STT en nube más ágil; Google Neural2 el TTS más veloz. Los IC95% abajo dicen si esas brechas se distinguen con $n=5$; no es un test de hipótesis formal.

Comparaciones entre proveedores

LLM: latencia - **groq** (695 ms, IC95% [507-884]) vs **openai** (1781 ms): $\Delta = 1086$ ms (2.6× más lento); los IC95% quedan separados; la diferencia parece real, no solo ruido. - **groq** (695 ms, IC95% [507-884]) vs **ollama** (36642 ms): $\Delta = 35947$ ms (52.7× más lento); los intervalos no se superponen; a ojo la diferencia importa.

STT: latencia - **deepgram** (428 ms, IC95% [355-502]) vs **openai** (2127 ms): $\Delta = 1699$ ms (5.0× más lento); los IC95% quedan separados; la diferencia parece real, no solo ruido. - **deepgram** (428 ms, IC95% [355-502]) vs **faster-whisper** (37482 ms): $\Delta = 37053$ ms (87.5× más lento); los IC95% quedan separados; la diferencia parece real, no solo ruido.

STT: WER (menor es mejor) - **assemblyai** (WER 0.0500, IC95% [0.0300-0.0700]) vs **faster-whisper** (WER 0.0500): $\Delta = +0.0000$; los intervalos se superponen; la diferencia podría deberse al tamaño de muestra.

TTS: latencia - **google** (300 ms, IC95% [225-375]) vs **azure** (849 ms): $\Delta = 549$ ms (2.8× más lento); los intervalos no se cruzan; con esta muestra la brecha es visible. - **google** (300 ms, IC95% [225-375]) vs **openai** (3369 ms): $\Delta = 3069$ ms (11.2× más lento); los IC95% quedan separados; la diferencia parece real, no solo ruido.

Variabilidad relativa (CV%)

El coeficiente de variación ($\text{desvío} \div \text{media} \times 100$) indica qué tan estables fueron las mediciones. Valores altos en nube suelen deberse a red o colas del proveedor; en local (Ollama, faster-whisper, Piper), a carga de GPU/RAM o a audios más largos. Los casos listados conviene contrastarlos con el p95 de la misma capa.

Variabilidad LLM (latencia) - **ollama**: CV = 91.5% (media 36642.46, σ = 33517.59) - **anthropic**: CV = 66.4% (media 6916.05, σ = 4591.55) - **groq**: CV = 65.6% (media 695.24, σ = 456.04) - **google**: CV = 54.3% (media 5928.33, σ = 3220.95)

Variabilidad STT (latencia) - **azure**: CV = 83.6% (media 6891.86, σ = 5764.40) - **deepgram**: CV = 71.6% (media 428.49, σ = 306.89) - **faster-whisper**: CV = 55.2% (media 37481.80, σ = 20692.11) - **openai**: CV = 51.0% (media 2127.13, σ = 1084.11)

Variabilidad TTS (latencia) - **azure**: CV = 74.3% (media 848.87, σ = 630.78) - **piper**: CV = 67.2% (media 3025.79, σ = 2032.30) - **google**: CV = 60.3% (media 300.05, σ = 181.00) - **elevenlabs**: CV = 58.0% (media 2147.46, σ = 1245.40)

Valores atípicos (criterio IQR $\times$ 1.5)

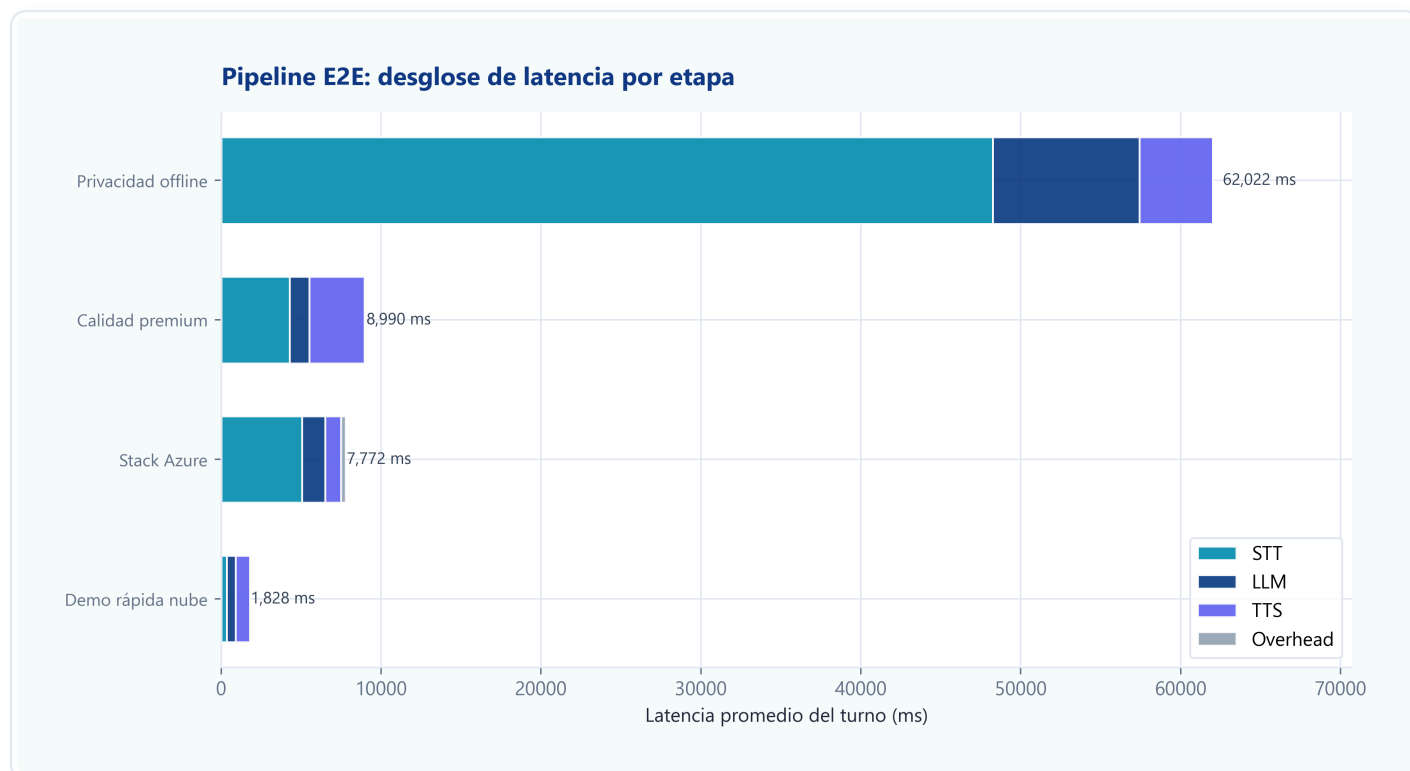
Cat.	Proveedor	Llamada	Valor	Cerca normal
llm	groq	p2_razonamiento run3	1646.68	[-362.45, 1480.98]
llm	groq	p2_razonamiento run4	1722.01	[-362.45, 1480.98]
llm	ollama	p2_razonamiento run5	158394.37	[-30118.65, 98685.67]
stt	assemblyai	a1_saludo_corto run1	5921.67	[3200.33, 4814.10]
stt	assemblyai	a2_oracion_media run1	5031.84	[3200.33, 4814.10]
stt	assemblyai	g2_fleurs run5	5833.86	[3200.33, 4814.10]
stt	assemblyai	g4_fleurs run4	5305.19	[3200.33, 4814.10]
stt	assemblyai	l4_noisy_largo run4	5803.41	[3200.33, 4814.10]
stt	assemblyai	l2_piper_largo run1	8319.27	[3200.33, 4814.10]
stt	assemblyai	l2_piper_largo run3	6992.67	[3200.33, 4814.10]
stt	assemblyai	l2_piper_largo run5	6932.94	[3200.33, 4814.10]
stt	deepgram	a1_saludo_corto run1	1080.76	[-91.75, 813.03]
stt	deepgram	a5_acentos_cr run2	941.11	[-91.75, 813.03]
stt	deepgram	a5_acentos_cr run5	1444.90	[-91.75, 813.03]
stt	deepgram	r1_cv_centroamerica run4	910.48	[-91.75, 813.03]
stt	deepgram	l2_piper_largo run1	1995.13	[-91.75, 813.03]
stt	deepgram	l2_piper_largo run2	895.77	[-91.75, 813.03]
stt	deepgram	l2_piper_largo run5	822.27	[-91.75, 813.03]
stt	faster-whisper	l1_fleurs_largo run5	132270.51	[-20367.03, 96115.73]
stt	openai	l1_fleurs_largo run2	4606.31	[-278.84, 4181.23]
stt	openai	l2_piper_largo run1	6534.24	[-278.84, 4181.23]
stt	openai	l2_piper_largo run3	4293.87	[-278.84, 4181.23]
stt	openai	l2_piper_largo run4	4348.61	[-278.84, 4181.23]
stt	openai	l2_piper_largo run5	4264.42	[-278.84, 4181.23]

Cat.	Proveedor	Llamada	Valor	Cerca normal
tts	azure	t3_parrafo_largo run1	3842.89	[443.47, 1006.86]
tts	elevenlabs	t3_parrafo_largo run1	4386.91	[480.70, 2940.26]
tts	elevenlabs	t3_parrafo_largo run2	4578.12	[480.70, 2940.26]
tts	elevenlabs	t3_parrafo_largo run3	4522.85	[480.70, 2940.26]
tts	elevenlabs	t3_parrafo_largo run4	4501.87	[480.70, 2940.26]
tts	elevenlabs	t3_parrafo_largo run5	4575.47	[480.70, 2940.26]
tts	google	t1_saludo_corto run1	698.95	[-110.48, 663.37]
tts	google	t3_parrafo_largo run1	843.77	[-110.48, 663.37]
tts	openai	t3_parrafo_largo run1	6234.49	[1085.08, 4670.41]
tts	openai	t3_parrafo_largo run2	5797.81	[1085.08, 4670.41]
tts	openai	t3_parrafo_largo run3	7647.38	[1085.08, 4670.41]
tts	openai	t3_parrafo_largo run4	5034.73	[1085.08, 4670.41]
tts	openai	t3_parrafo_largo run5	5475.83	[1085.08, 4670.41]
tts	piper	t3_parrafo_largo run5	7992.19	[-2410.20, 7514.73]

Los valores atípicos siguen en el promedio; se listan aparte. Cruzar con el desvío y el p95 en Secciones 3.1-3.3.

3.0.8 Pipeline end-to-end (STT → LLM → TTS)

Un **turno E2E** encadena en orden: audio del usuario → STT → LLM (con la transcripción) → TTS (con la respuesta). La **latencia total** es el tiempo de pared de punta a punta; el **overhead** es lo que queda fuera de STT + LLM + TTS (preparación, buffers y pasos entre etapas).



Desglose de latencia E2E por etapa

Figura: tiempo medio por etapa (STT, LLM, TTS y overhead) en los cuatro escenarios E2E. La barra total no coincide con la suma de medias de Sección 3.0.6 porque aquí cada etapa usa la salida real de la anterior.

Cobertura del batch E2E

Turnos registrados: 100/100 exitosos..

Sin turnos con error en el último batch.

Resumen por escenario (medido)

escenario	STT	LLM	TTS	N	STT ms	LLM ms	TTS ms	Overhead ms	Total ms	STT %	LLM %	TTS %	USD turno
Calidad premium	assemblyai	openai	elevenlabs	25	4277.38	1242.44	3450.55	20.03	8990.4	47.6	13.8	38.4	0.09
Demo rápida nube	deepgram	groq	google	25	344.18	568.47	877.82	37.75	1828.22	18.8	31.1	48	0.01
Privacidad offline	faster-whisper	ollama	pipec	25	48267	9167.73	4585.63	1.24	62021.6	77.8	14.8	7.4	0
Stack Azure	azure	openai	azure	25	5050.09	1449.2	992.56	280.08	7771.94	65	18.6	12.8	0.01

Tiempos medidos en la corrida: Calidad premium ≈ 9.0 s y ~\$0.0900/turno; Demo rápida nube ≈ 1.8 s y ~\$0.0100/turno; Privacidad offline ≈ 62.0 s y ~\$0.0000/turno; Stack Azure ≈ 7.8 s y ~\$0.0100/turno.

IC95% latencia total E2E por escenario

provider	n	mean	std	cv_pct	ci95_low	ci95_high
Demo rápida nube	25	1828.22	670.6	36.7	1551.24	2105.2
Stack Azure	25	7771.94	6287.63	80.9	5174.9	10369
Calidad premium	25	8990.4	1847.16	20.5	8227.45	9753.35
Privacidad offline	25	62021.6	38217.2	61.6	46236.4	77806.9

E2E ESCENARIOS: latencia - **Demo rápida nube** (1828 ms, IC95% [1551-2105]) vs **Stack Azure** (7772 ms): Δ = 5944 ms (4.3× más lento); los IC95% quedan separados; la diferencia parece real, no solo ruido. - **Demo rápida nube** (1828 ms, IC95% [1551-2105]) vs **Privacidad offline** (62022 ms): Δ = 60193 ms (33.9× más lento); sin solapamiento entre intervalos en esta corrida.

Suma de componentes vs medición E2E

La suma de medias de Sección 3.0.6 **no coincide** con el turno real: en E2E cada etapa usa la salida de la anterior y comparte la misma red o GPU.

escenario	Suma componentes (ms)	E2E medido (ms)	Δ E2E – suma	Δ %
Demo rápida nube	1423.8	1828.22	404.4	28.4

escenario	Suma componentes (ms)	E2E medido (ms)	Δ E2E – suma	Δ %
Calidad premium	8146.5	8990.4	843.9	10.4
Privacidad offline	77150.1	62021.6	-15128.5	-19.6
Stack Azure	9521.7	7771.94	-1749.8	-18.4

Desglose por audio de prueba

escenario	test_id	total_ms	n
Calidad premium	a1_saludo_corto	6953.7	5
Calidad premium	a2_oracion_media	10055.4	5
Calidad premium	c1_cv_noisy	7273.2	5
Calidad premium	g1_fleurs	10273.6	5
Calidad premium	l1_fleurs_largo	10396.1	5
Demo rápida nube	a1_saludo_corto	2030.4	5
Demo rápida nube	a2_oracion_media	1839.2	5
Demo rápida nube	c1_cv_noisy	1686.4	5
Demo rápida nube	g1_fleurs	1809.7	5
Demo rápida nube	l1_fleurs_largo	1775.3	5
Privacidad offline	a1_saludo_corto	55912	5
Privacidad offline	a2_oracion_media	70367.9	5
Privacidad offline	c1_cv_noisy	34576.6	5
Privacidad offline	g1_fleurs	35719.2	5
Privacidad offline	l1_fleurs_largo	113532	5
Stack Azure	a1_saludo_corto	3691	5
Stack Azure	a2_oracion_media	6126.5	5
Stack Azure	c1_cv_noisy	4147.5	5
Stack Azure	g1_fleurs	4956.8	5
Stack Azure	l1_fleurs_largo	19937.9	5

Por audio: en nube, los saludos cortos (a1) suelen ser más rápidos que FLEURS (g1); en offline, la oración media (a2) fue la más lenta por la carga sostenida de la GPU.

Overhead medio: 84.8 ms (IC95% [52.7-116.9]). Tiempo que el usuario percibe pero no se atribuye a una sola API.

3.0.9 Profundidad: dimensiones 4, 5 y 6

Notas ampliadas por servicio: las tablas de arriba resumen puntuación y métricas de este repositorio; abajo están las fichas con políticas, parámetros de API y ejemplos de código (documentación consultada en mayo 2026).

Los puntajes cualitativos se contrastaron con términos de servicio públicos.

Dimensión 4: Privacidad y gobernanza

servicio	score	análisis
llm/anthropic	4	Nube, con política explícita de no usar prompts para entrenar; mejor que un free tier genérico, pero el dato igual viaja.
llm/google	3	Gemini pasa por Google Cloud; en plan de pago hay más control que en el tier gratis (revisar consola del proyecto).
llm/groq	4	Nube acelerada; declaran no entrenar con las llamadas. Para datos delicados haría falta contrato, no solo API key.
llm/ollama	5	Todo en el laptop; el prompt no sale a internet.
llm/openai	3	Texto y audio salen del equipo hacia OpenAI; la política API dice que no entrenan por defecto, pero sigue siendo nube.
stt/assemblyai	3	Nube; retención según el plan que contrates.
stt/azure	3	Datos en la región Azure que elijas; orientado a clientes con requisitos de cumplimiento.
stt/deepgram	3	Audio en nube; en contratos grandes se puede negociar retención mínima.
stt/faster-whisper	5	El audio no sale del equipo.
stt/openai	3	Subo el WAV a OpenAI; misma política de datos que el resto de la API.
tts/azure	3	Síntesis en la región Azure elegida.
tts/elevenlabs	3	Nube; clonar voz implica datos biométricos y consentimiento.
tts/google	3	Texto procesado en infra de Google; hace falta proyecto GCP.
tts/openai	3	El texto va a OpenAI para sintetizar; misma lógica de privacidad que el LLM.
tts/piper	5	Offline; el texto no se manda a terceros.

Dimensión 5: Customización

servicio	score	análisis
llm/anthropic	5	Buen margen en instrucciones largas, tools y cambiar entre modelos Claude.
llm/google	4	Instrucciones, filtros y parámetros sí; subir un modelo propio por API, no.
llm/groq	3	Pocos modelos publicados (Llama, Mixtral) y parámetros básicos; sin tools finas ni entrenar uno propio.
llm/ollama	4	Se cambia de modelo (Llama, Mistral), Modelfile y parámetros locales; no hay fine-tuning en nube.
llm/openai	5	El más flexible del grupo en LLM: system prompt, temperatura, varios GPT y modo JSON.
stt/assemblyai	4	Vocabulario custom, speaker labels y extras (resúmenes) en la misma API. WER medio 4.7 % en 14 audios.
stt/azure	4	Modelos custom, frases guía y pronunciación; hay locale es-CR. WER medio 7.1 % en 14 audios.
stt/deepgram	4	Varios Nova, idiomas, puntuación y diarización si pagás plan alto. WER medio 6.6 % en 14 audios.

servicio	score	análisis
stt/faster-whisper	3	Elegís tamaño de modelo y GPU/CPU; no hay portal para entrenar con tus datos. WER medio 5.0 % en 14 audios.
stt/openai	3	Idioma y un prompt de contexto; Whisper no se adapta fácil con datos propios. WER medio 6.5 % en 14 audios.
tts/azure	4	Voces neurales, SSML y voz custom en planes altos.
tts/elevenlabs	5	Lo más flexible en voz: clonación, estilos y control expresivo (5/5 en escucha, Sección 3.0.5).
tts/google	4	Catálogo grande Neural2/Wavenet, SSML, velocidad y tono.
tts/openai	4	Varias voces y estilo en los modelos de voz recientes.
tts/piper	2	Solo voces ONNX pregrabadas; sin clonación ni matices de emoción.

Dimensión 6: Facilidad de integración

servicio	score	análisis
llm/anthropic	5	Paquete <code>anthropic</code> en Python; el stream de mensajes anduvo sin sorpresas en las pruebas.
llm/google	4	SDK <code>google-generativeai</code> ; tiene sentido si el proyecto ya vive en GCP.
llm/groq	4	API casi compatible con OpenAI: cambié URL y clave y corrió; fue el LLM más rápido aquí.
llm/ollama	3	HTTP en localhost; hay que tener el daemon arriba y no hay SLA de nadie.
llm/openai	5	SDK de Python con streaming; en este repo la misma cuenta cubre LLM, STT y TTS.
stt/assemblyai	4	SDK Python y REST claros para mandar audio y leer la transcripción.
stt/azure	4	SDK en Python y C#; lo usé pensando en Unity/.NET.
stt/deepgram	4	SDK reciente y WebSocket para vivo; en esta corrida fue el STT en nube más rápido.
stt/faster-whisper	3	Librería Python + CUDA; en el Quadro P2000 la latencia fue alta.
stt/openai	5	Una API key con LLM y TTS; pocas líneas de código para transcribir.
tts/azure	4	Misma clave que Azure STT; SDK o REST.
tts/elevenlabs	4	REST/SDK orientados a producto de voz; aquí fue el TTS más caro por carácter.
tts/google	4	Cliente oficial; hay que activar API y credenciales en la consola.
tts/openai	5	Misma API key que LLM y STT; devuelve audio directo.
tts/piper	2	Ejecutable o subprocess; no hay SDK cloud listo.

Fichas por servicio

llm/openai

Privacidad (3/5): Cloud con políticas API claras pero dependencia de tercero y retención acotada. - OpenAI indica que los datos enviados vía API no se usan para entrenar modelos por defecto. - Retención temporal para abuso o monitorización; en planes empresariales se puede pedir retención cero. - El audio y el texto salen del equipo del usuario hacia la nube. Documentación: [OpenAI API Data Usage Policies](#) (2026-05-20).

Customización (5/5): Muchas opciones en la API: instrucciones de sistema, herramientas y varios modelos. - Parámetros: temperatura, top_p, máximo de tokens, respuestas en JSON. - Llamadas a herramientas externas para acciones del agente. Documentación: [OpenAI API Reference: Chat Completions](#) (2026-05-20).

Integración (5/5): SDK estable, streaming SSE y el mismo proveedor cubre STT/TTS en este repo. - Cliente oficial `openai` con streaming token a token. - Compatible con proxies y gateways; documentación extensa. Documentación: [OpenAI Python SDK](#) (2026-05-20).

```
from openai import OpenAI
client = OpenAI
stream = client.chat.completions.create(
    model='gpt-4o', messages=[ ... ], stream=True)
```

llm/anthropic

Privacidad (4/5): Política API que promete no entrenar con prompts; mejor que free tiers sueltos. - Anthropic comunica que las solicitudes API no alimentan entrenamiento por defecto. - Opciones de contrato para retención y cumplimiento. - Sigue siendo procesamiento en nube US/EU según región. Documentación: [Anthropic Privacy Policy / Commercial Terms](#) (2026-05-20).

Customización (5/5): Instrucciones largas, herramientas y familia Claude con ventanas extensas. - Claude Sonnet/Opus intercambiables; instrucciones de sistema separadas. - Uso de herramientas y respuestas en JSON en modelos recientes. Documentación: [Anthropic Messages API](#) (2026-05-20).

Integración (5/5): SDK Python oficial; streaming de mensajes bien soportado. - Paquete `anthropic` con tipado y manejo de eventos stream. - API REST estable; ejemplos oficiales para agentes. Documentación: [Anthropic Python SDK](#) (2026-05-20).

```
import anthropic
client = anthropic.Anthropic
client.messages.stream(model='claude-sonnet-4-6', max_tokens=512, messages=[ ... ])
```

llm/google

Privacidad (3/5): Gemini API en GCP: buen control empresarial, pero free tier con reglas distintas. - En tier de pago los datos no deberían usarse para mejorar productos sin consentimiento. - Free tier históricamente

más permisivo; verificar consola del proyecto. Documentación: [Google AI Gemini API: Billing & data](#) (2026-05-20).

Customización 4/5: Instrucciones, filtros y parámetros sí; subir un modelo propio por API, no. Integración 4/5: SDK [google-genai](#) ; tiene sentido si el proyecto ya vive en GCP.

llm/groq

Integración (4/5): Compatible con clientes estilo OpenAI; en el benchmark fue el LLM más rápido. - Base URL + API key; ideal para cambiar solo el endpoint. - Streaming soportado; SDK oficial Python. Documentación: [Groq Python SDK](#) (2026-05-20).

Privacidad 4/5: Nube acelerada; declaran no entrenar con las llamadas. Para datos delicados haría falta contrato, no solo API key. Customización 3/5: Pocos modelos publicados (Llama, Mixtral) y parámetros básicos; sin tools finas ni entrenar uno propio.

```
from groq import Groq
client = Groq
client.chat.completions.create(model='llama-3.3-70b-versatile', messages=[ ... ], stream=True)
```

llm/ollama

Corre en la máquina con Ollama ([llm/ollama](#)). Privacidad 5/5 porque el texto no sale del equipo; integración 3/5 por el HTTP local sin SLA de nube.

Customización 4/5: Se cambia de modelo (Llama, Mistral), Modelfile y parámetros locales; no hay fine-tuning en nube.

stt/deepgram

Integración (4/5): SDK actual y WebSocket adecuado para audio en vivo. - REST para batch; WS para micrófono en vivo. - Buena latencia medida en este benchmark. Documentación: [Deepgram Python SDK](#) (2026-05-20).

Privacidad 3/5: Audio en nube; en contratos grandes se puede negociar retención mínima. Customización 4/5: Varios Nova, idiomas, puntuación y diarización si pagás plan alto.

```
client.listen.v1.media.transcribe_file(
    request=audio_bytes, model='nova-3', language='es', smart_format=True)
```

stt/openai

Integración (5/5): Misma API key que LLM/TTS; multipart simple. - Endpoint /v1/audio/transcriptions. - Ejemplos oficiales en Python y curl. Documentación: [OpenAI Audio transcriptions](#) (2026-05-20).

Privacidad 3/5: Subo el WAV a OpenAI; misma política de datos que el resto de la API. Customización 3/5: Idioma y un prompt de contexto; Whisper no se adapta fácil con datos propios.

`stt/assemblyai`

Customización (4/5): Vocabulario propio, etiquetas de hablante y funciones extra de resumen. - Lista de palabras priorizadas para términos del dominio. - Modelos universales con opciones de formato de salida. Documentación: [AssemblyAI Custom vocabulary](#) (2026-05-20).

Privacidad 3/5: Nube; retención según el plan que contrates. Integración 4/5: SDK Python y REST claros para mandar audio y leer la transcripción.

`stt/azure`

Privacidad (3/5): Datos en región Azure elegida; pensado para clientes con auditorías. - No entrenamiento con datos del cliente en Speech estándar. - Requiere suscripción y logging configurable. Documentación: [Microsoft Azure Speech: Data privacy](#) (2026-05-20).

Customización (4/5): Modelos de voz personalizados, frases y ajuste de pronunciación. - Modelos base + adaptación con datos propios. - Locale es-CR disponible. Documentación: [Azure Custom Speech](#) (2026-05-20).

Integración (4/5): SDK Azure + REST; sirve en proyectos.NET o Unity. - `azure-cognitiveservices-speech` en Python/C#. - Misma key para TTS neural. Documentación: [Azure Speech SDK](#) (2026-05-20).

```
speech_config = speechsdk.SpeechConfig(subscription=key, region=region)
recognizer = speechsdk.SpeechRecognizer(speech_config=speech_config)
```

`stt/faster-whisper`

faster-whisper transcribe sin API (`stt/faster-whisper`). El audio no se sube a internet (privacidad 5/5). En esta corrida la latencia fue alta en el Quadro P2000.

Customización 3/5: Elegís tamaño de modelo y GPU/CPU; no hay portal para entrenar con tus datos.

`tts/google`

Customización (4/5): Amplio catálogo Neural2/Wavenet y SSML. - Voz es-ES-Neural2-A usada en el benchmark. - Ajuste de velocidad de habla y tono. Documentación: [Google Cloud TTS voices](#) (2026-05-20).

Privacidad 3/5: Texto procesado en infra de Google; hace falta proyecto GCP. Integración 4/5: Cliente oficial; hay que activar API y credenciales en la consola.

tts/openai

Integración (5/5): Misma cuenta/API que LLM/STT. - Endpoint audio.speech. - Respuesta MP3/PCM directa. Documentación: [OpenAI Audio speech](#) (2026-05-20).

Privacidad 3/5: El texto va a OpenAI para sintetizar; misma lógica de privacidad que el LLM. Customización 4/5: Varias voces y estilo en los modelos de voz recientes.

```
client.audio.speech.create(  
    model='gpt-4o-mini-tts', voice='alloy', input='Hola mundo')
```

tts/elevenlabs

Privacidad (3/5): Cloud; clonación implica voz biométrica y consentimiento. - Políticas de retención según plan. - No usar con datos personales sin revisar el contrato de tratamiento. Documentación: [ElevenLabs Privacy](#) (2026-05-20).

Customización (5/5): Clonación, estilos y la voz más expresiva en la escucha manual (Sección 3.0.5). - Clonación de voz instantánea en planes superiores. - Modelo multilingüe v2 para español. Documentación: [ElevenLabs API](#) (2026-05-20).

Integración (4/5): REST + SDK; pensado para apps de voz. - Streaming de audio. - Mayor costo por carácter en este benchmark. Documentación: [ElevenLabs Python SDK](#) (2026-05-20).

```
client.text_to_speech.convert(  
    voice_id=' ... ', text=' ... ', model_id='eleven_multilingual_v2')
```

tts/azure

Privacidad (3/5): Región Azure; cumplimiento SOC/ISO en suscripción. - Voz neural procesada en región elegida. - Logs configurables. Documentación: [Azure AI Speech](#) (2026-05-20).

Customización (4/5): Voces neurales, marcado SSML y voz neural personalizada en planes altos. - es-CR-MariaNeural para costarricense. - Prosodia vía SSML. Documentación: [Azure Neural TTS](#) (2026-05-20).

Integración (4/5): SDK + SSML; par con Azure STT. - Una key para STT+TTS. - REST batch disponible. Documentación: [Azure Speech SDK: TTS](#) (2026-05-20).

```
synthesizer = speechsdk.SpeechSynthesizer(speech_config, audio_config)  
synthesizer.speak_text_async('Hola').get
```

tts/piper

Piper sintetiza offline ([tts/piper](#)). No hay costo por carácter ni envío de texto a terceros. La voz suena más sintética que ElevenLabs o Azure; integración 2/5 por el binario y subprocess.

Customización 2/5: Solo voces ONNX pregrabadas; sin clonación ni matices de emoción.

```
# ejemplo: piper -m modelo.onnx -f salida.wav < texto.txt
```

Contraste entre proveedores

En privacidad, lo que separa a Ollama, faster-whisper y Piper del resto es que no mandan audio ni texto a un tercero. Anthropic y Groq declaran no entrenar con las llamadas API, pero el procesamiento sigue siendo en nube. En customización, OpenAI y Anthropic dejan más margen en LLM (prompt, tools, formatos). En voz, ElevenLabs fue la referencia en la escucha del informe. Groq y Piper tienen menos voces y modelos intercambiables. En integración, usar OpenAI para las tres capas reduce cableado (una API key). Azure Speech le cae bien si el cliente ya es.NET/Unity. Los locales funcionan, pero piden GPU, binarios y mantenimiento sin SLA de proveedor.

3.1 LLM

Promedio de 5 corridas por input (solo llamadas exitosas). N = total agregado; TTFT solo en LLM. Gráficos y comentarios de esta capa van después.

Proveedor	Modelo	Despliegue	N	Lat. prom.	Lat. σ	Lat. p95	TTFT	USD/llam.	Cat.
anthropic	claude-sonnet-4-6	cloud	25	6916.05	4591.55	14611.1	1346.41	0.005728	llm
google	gemini-2.5-flash	cloud	25	5928.33	3220.95	11644.5	4832.61	0.000779	llm
groq	llama-3.3-70b-versatile	cloud	25	695.24	456.04	1610.71	226.99	0.000226	llm
ollama	llama3.2:3b	local	25	36642.5	33517.6	76198.6	1740.66	0	llm
openai	gpt-4o	cloud	25	1780.93	864.34	3365.36	584.04	0.001979	llm

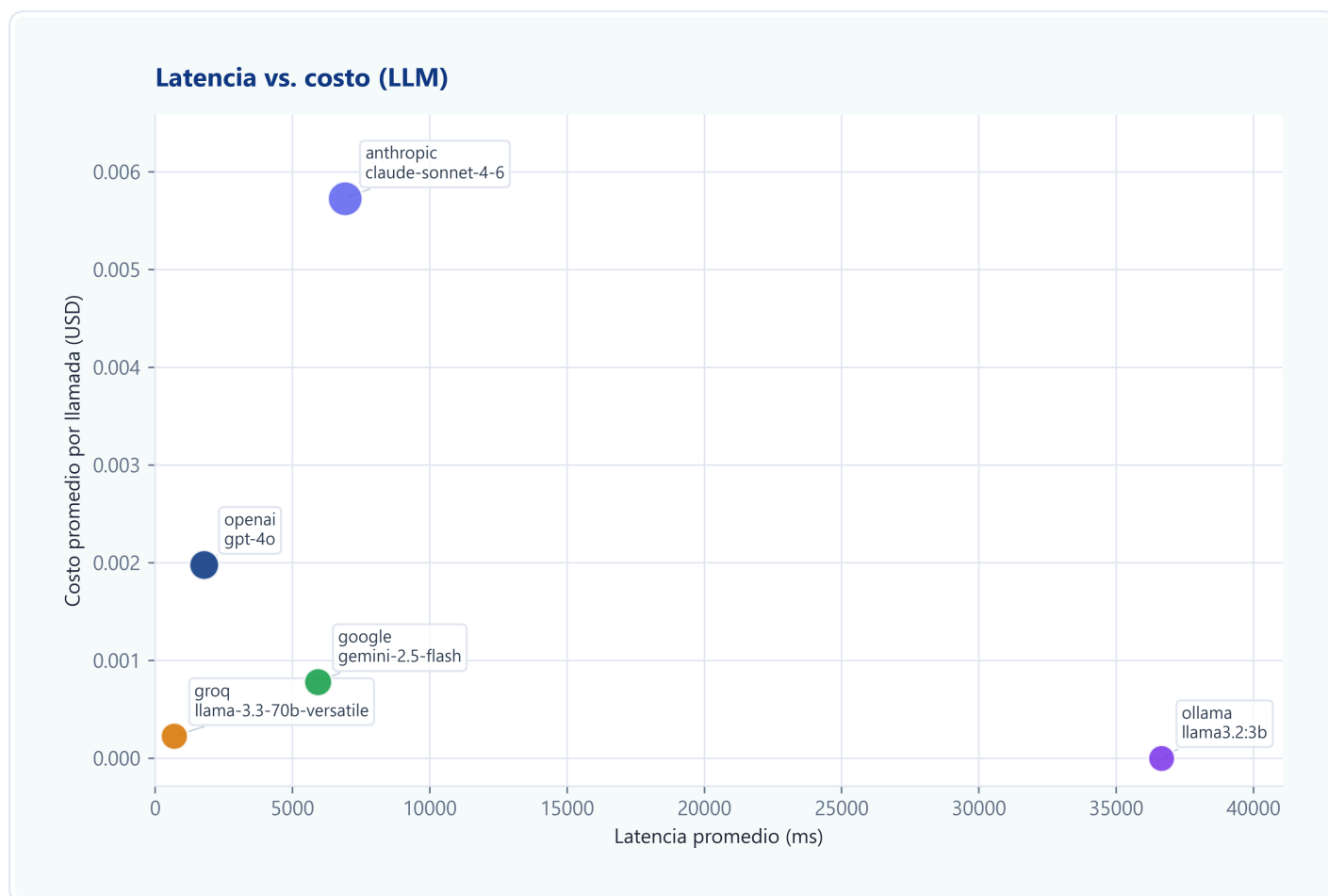
Matriz 5×6 (LLM)

Desglose de la misma matriz que en Sección 3.0.2, junto a los gráficos de esta capa.

Desglose de la matriz maestra por capa del pipeline (LLM, STT, TTS).

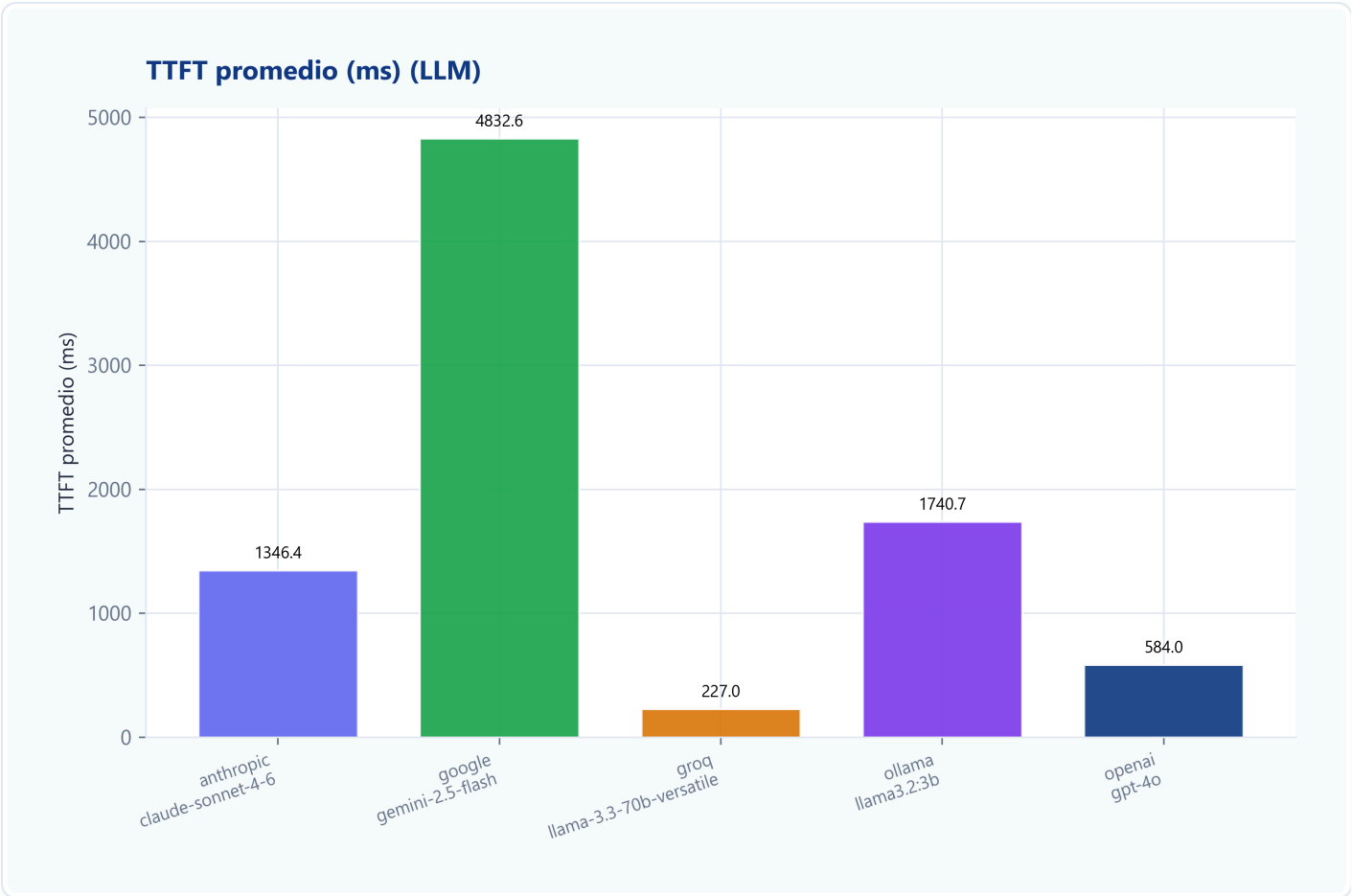
LLM

Prov.	Lat. (ms)	TTFT	WER	USD	Priv.	Cust.	Integr.	Precisión
anthropic	6916.0000	1346.4000	nan	0.0057	4	5	5	Coherencia (cual.)
google	5928.3000	4832.6000	nan	0.0008	3	4	4	Coherencia (cual.)
groq	695.2000	227.0000	nan	0.0002	4	3	4	Coherencia (cual.)
ollama	36642.5000	1740.7000	nan	0.0000	5	4	3	Coherencia (cual.)
openai	1780.9000	584.0000	nan	0.0020	3	5	5	Coherencia (cual.)



Latencia vs costo (LLM)

Figura: cada punto es un proveedor LLM; eje X = latencia media (ms), eje Y = costo por llamada (USD).



TTFT (LLM)
Figura: tiempo al primer token (TTFT) en streaming; solo aplica a LLM.

Comentarios (LLM). Groq fue el más rápido y barato; sus intervalos de confianza no se cruzan con los de OpenAI (ver Sección 3.0.7). OpenAI GPT-4o equilibra velocidad y calidad de razonamiento. Gemini Flash responde bien en español, pero el TTFT alto lo hace poco adecuado para demos en vivo. Ollama no cobra API, pero en este equipo es demasiado lento para tiempo real. La coherencia e instrucciones complejas se documentan en Sección 3.0.4.

3.2 STT

Promedio de 5 corridas por input (solo llamadas exitosas). N = total agregado; WER solo en STT. Gráficos y comentarios de esta capa van después.

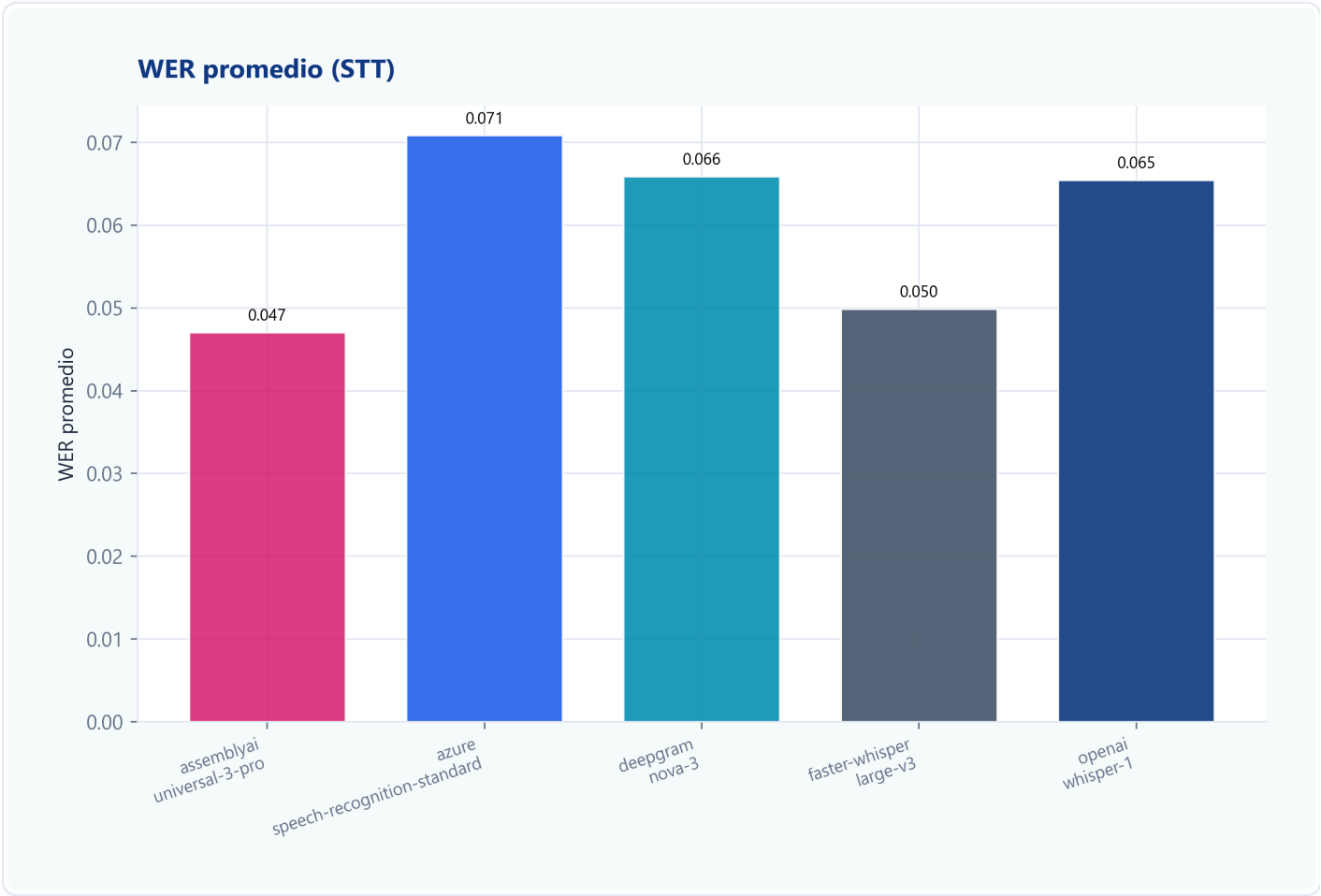
Proveedor	Modelo	Despliegue	N	Lat. prom.	Lat. σ	Lat. p95	USD/llam.	WER	Cat.
assemblyai	universal-3-pro	cloud	70	4218.13	841.57	5882.15	0.001064	0.05	stt
azure	speech-recognition-standard	cloud	70	6891.86	5764.4	16604.3	0.004938	0.07	stt
deepgram	nova-3	cloud	70	428.49	306.89	927.33	0.001271	0.07	stt
faster-whisper	large-v3	local	70	37481.8	20692.1	69884.5	0	0.05	stt
openai	whisper-1	cloud	70	2127.13	1084.11	4280.62	0.001774	0.07	stt

Matriz 5×6 (STT)

Desglose de la misma matriz que en Sección 3.0.2, junto a los gráficos de esta capa.

STT

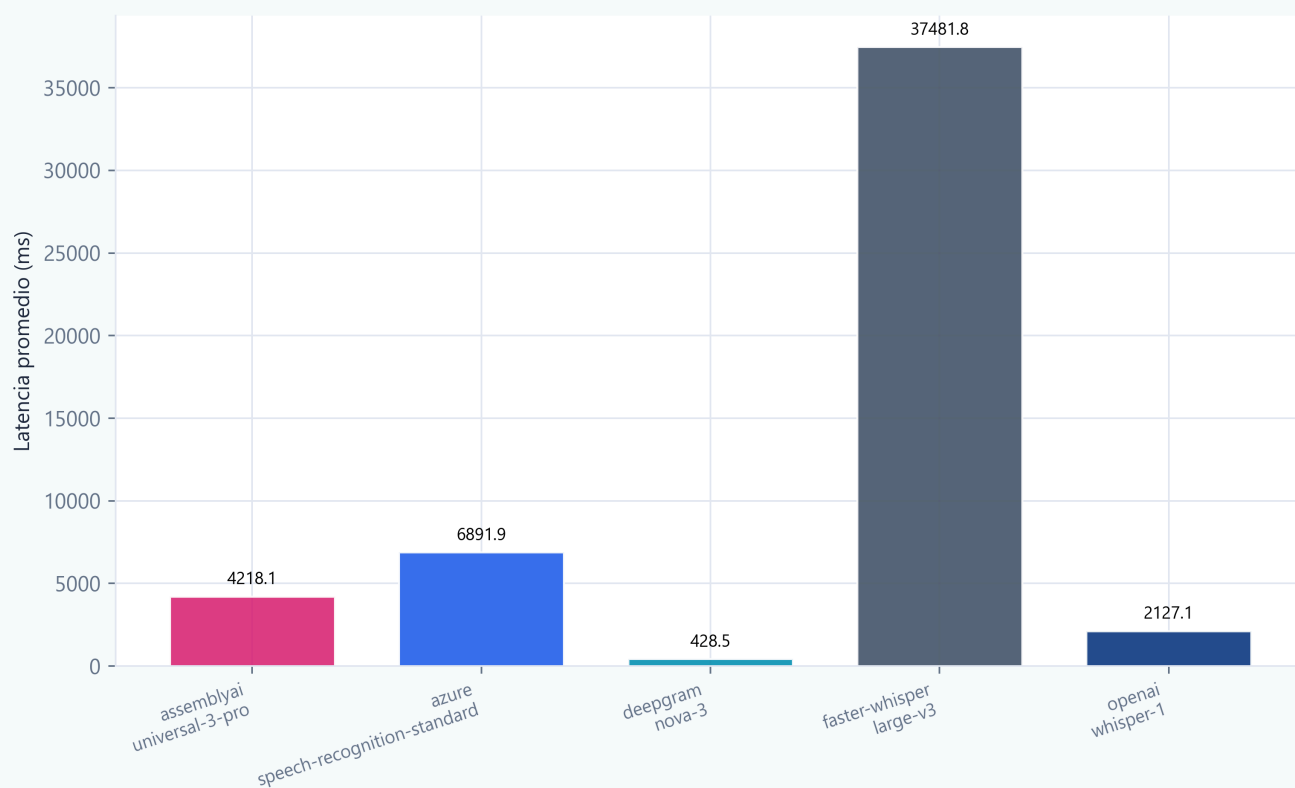
Prov.	Lat. (ms)	TTFT	WER	USD	Priv.	Cust.	Integr.	Precisión
assemblyai	4218.1000	nan	0.0471	0.0011	3	4	4	WER=0.0471
azure	6891.9000	nan	0.0709	0.0049	3	4	4	WER=0.0709
deepgram	428.5000	nan	0.0659	0.0013	3	4	4	WER=0.0659
faster-whisper	37481.8000	nan	0.0499	0.0000	5	3	3	WER=0.0499
openai	2127.1000	nan	0.0655	0.0018	3	3	5	WER=0.0655



WER (STT)

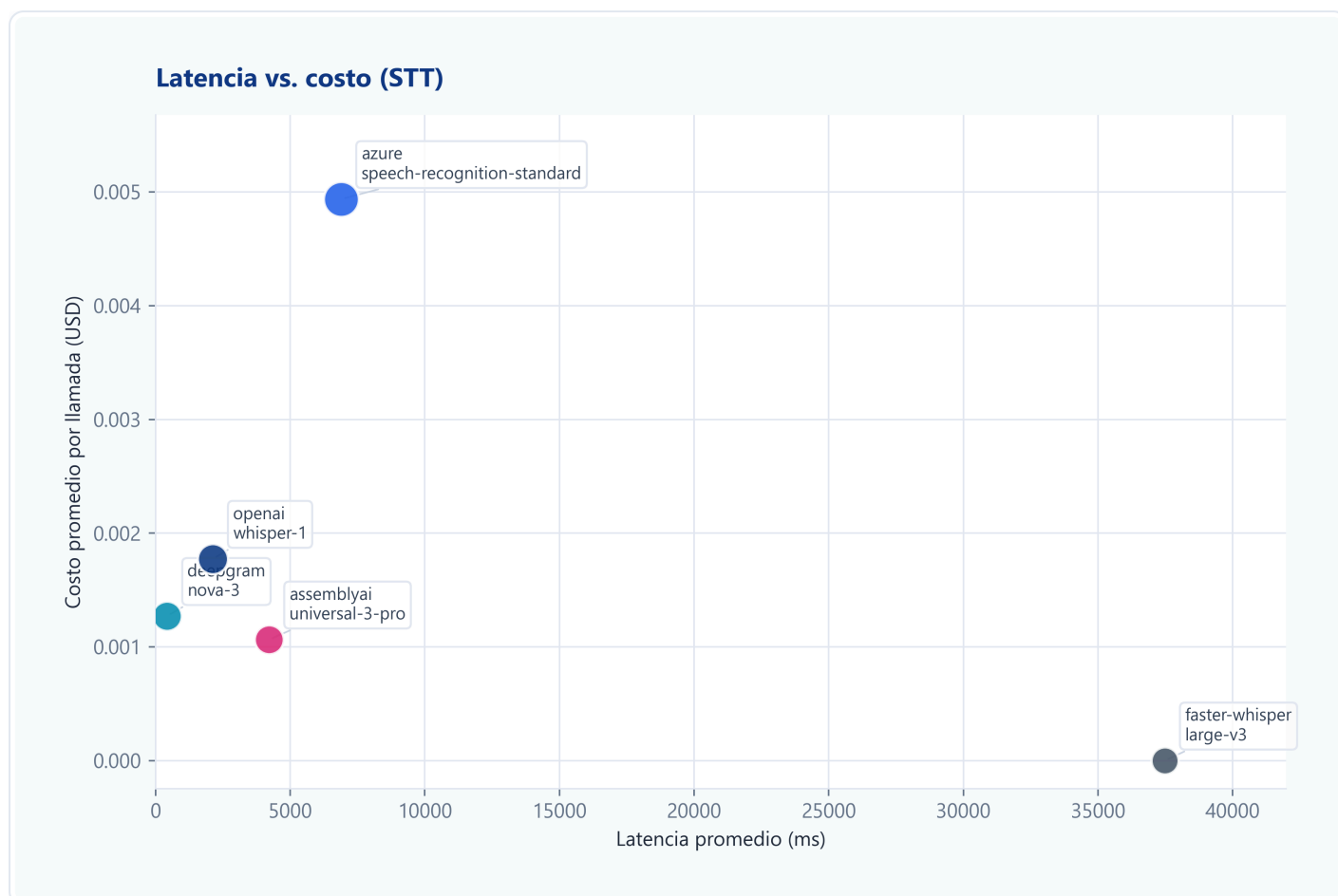
Figura: WER promedio global por proveedor (todas las fuentes de audio mezcladas).

Latencia promedio (ms) (STT)



Latencia (STT)

Figura: latencia media por proveedor STT (ms).



Latencia vs costo (STT)

Figura: relación latencia y costo por proveedor STT.

Comentarios (STT). Deepgram fue el STT en nube más rápido; AssemblyAI y Azure cuestan más por llamada sin ganar tanto en tiempo. El WER mejora con voz humana FLEURS (~0,03) que con audio sintético Piper (~0,14-0,19), así que los resultados con [a1](#) - [a5](#) son conservadores frente a usuarios reales. faster-whisper local tiene WER similar sin pagar API, pero tardó ~39 s por clip en este hardware. El desglose por fuente de audio está en Sección 3.0.3.

3.3 TTS

Promedio de 5 corridas por input (solo llamadas exitosas). N = total agregado. Gráficos y comentarios de esta capa van después.

Proveedor	Modelo	Despliegue	N	Lat. prom.	Lat. σ	Lat. p95	USD/llam.	Cat.
azure	es-CR-MariaNeural	cloud	25	848.87	630.78	955.76	0.00273	tts
elevenlabs	eleven_multilingua_l_v2	cloud	25	2147.46	1245.4	4564.95	0.05118	tts
google	es-ES-Neural2-A	cloud	25	300.05	181	676.96	0.00273	tts
openai	gpt-4o-mini-tts	cloud	25	3368.56	1483.2	6147.16	0.10236	tts

Proveedor	Modelo	Despliegue	N	Lat. prom.	Lat. σ	Lat. p95	USD/llam.	Cat.
piiper	es_ES-davefx-medium	local	25	3025.79	2032.3	7158.81	0	tts

Matriz 5×6 (TTS)

Desglose de la misma matriz que en Sección 3.0.2, junto a los gráficos de esta capa.

TTS

Prov.	Lat. (ms)	TTFT	WER	USD	Priv.	Cust.	Integr.	Precisión
azure	848.9000	nan	nan	0.0027	3	4	4	MOS 1-5 (escucha manual)
elevenlabs	2147.5000	nan	nan	0.0512	3	5	4	MOS 1-5 (escucha manual)
google	300.1000	nan	nan	0.0027	3	4	4	MOS 1-5 (escucha manual)
openai	3368.6000	nan	nan	0.1024	3	4	5	MOS 1-5 (escucha manual)
piiper	3025.8000	nan	nan	0.0000	5	2	2	MOS 1-5 (escucha manual)



Latencia (TTS)

Figura: latencia media por proveedor TTS (ms).



Latencia vs costo (TTS)

Figura: relación latencia y costo por proveedor TTS.

Comentarios (TTS). Google TTS Neural2 fue el más rápido con costo razonable. Piper no cobra API pero suena más robótico (Sección 3.0.5). ElevenLabs tuvo la mejor naturalidad, con un costo por carácter ~19× mayor que Google en esta corrida. OpenAI TTS es cómodo si el LLM ya es de OpenAI. Las notas de escucha manual por proveedor se registran en [data/tts_mos_scores.json](#).

3.4 Análisis cualitativo: dimensiones 4, 5 y 6

Las dimensiones 4-6 se puntuaron del 1 al 5 revisando políticas públicas y lo que permite cada API (consulta mayo 2026). El detalle largo está en Sección 3.0.9.

Rúbrica de evaluación

Dimensión 4: Privacidad

- **5/5:** Procesamiento 100 % local o sin conexión a internet; sin envío de contenido a terceros.
- **4/5:** Nube con política explícita de no entrenamiento y opciones de retención mínima.

- **3/5:** Nube estándar por API; los datos salen del dispositivo pero con políticas publicadas.
- **2/5:** Nube con retención amplia, plan gratuito permisivo o uso comercial de datos.
- **1/5:** Sin política clara; alto riesgo de reentrenamiento o almacenamiento prolongado.

Dimensión 5: Customización

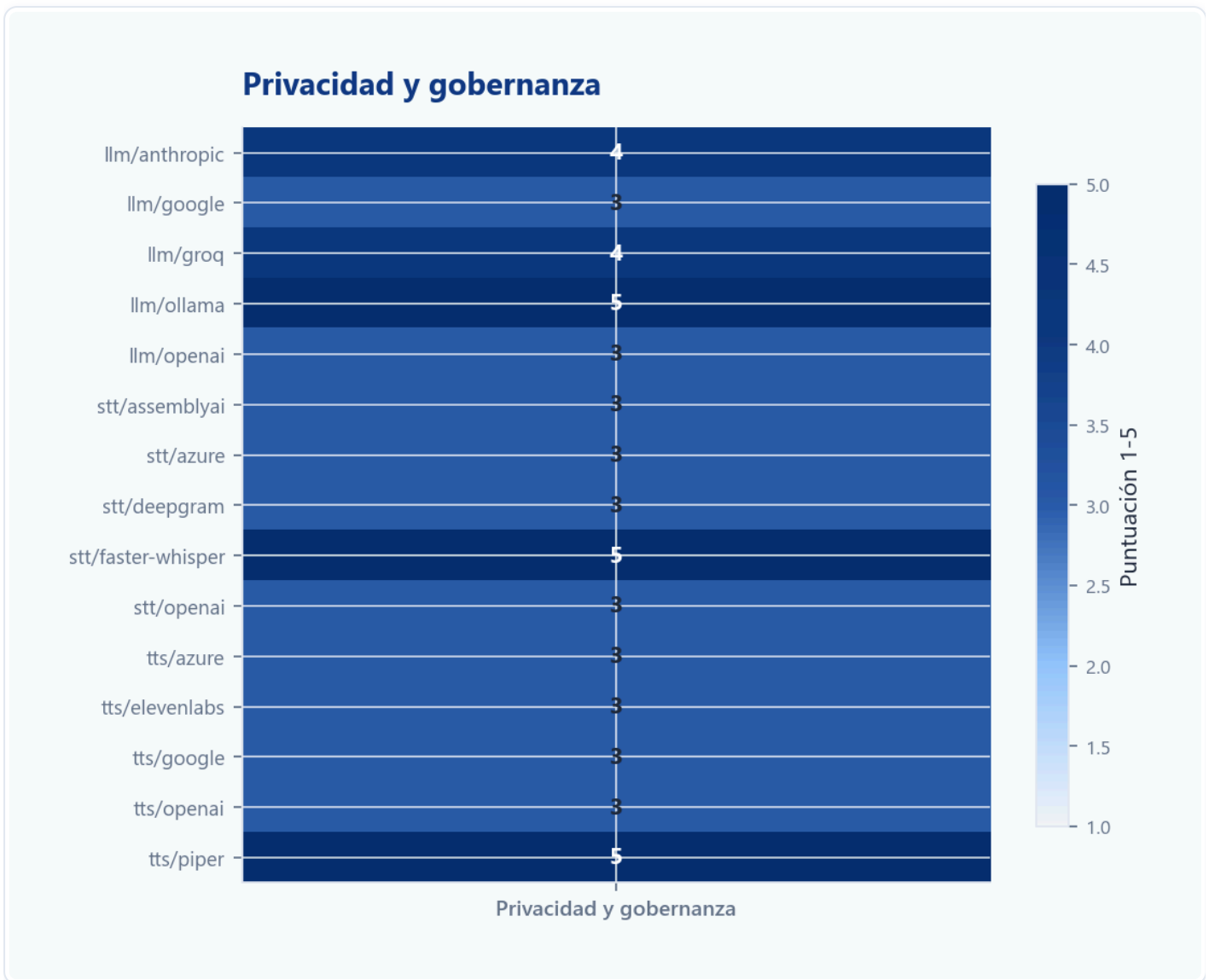
- **5/5:** Clonación de voz, herramientas externas, instrucciones de sistema avanzadas o adaptación de modelo.
- **4/5:** Marcado de voz (SSML), varias voces, vocabulario propio o modelos intercambiables (Modelfile).
- **3/5:** Parámetros básicos (idioma, temperatura, tamaño de modelo).
- **2/5:** Solo cambio de voz o modelo preentrenado; pocos controles de ajuste.
- **1/5:** Servicio cerrado sin opciones de ajuste relevantes.

Dimensión 6: Integración

- **5/5:** SDK oficial con streaming; misma cuenta cubre varias APIs del pipeline.
- **4/5:** SDK o REST bien documentado; WebSocket donde aplica.
- **3/5:** Librería o HTTP local; requiere ajuste operativo (GPU, binarios).
- **2/5:** CLI/subprocess o cableado manual sin SDK unificado.
- **1/5:** Sin API estable; integración poco documentada.

Lectura de los mapas de calor: cada fila es un servicio; el número en la celda es la puntuación 1-5 de la rúbrica (barra lateral: 1 = peor, 5 = mejor).

Dimensión 4: Privacidad y gobernanza



Dimensión 4: Privacidad y gobernanza

Figura: mapa de calor (Privacidad y gobernanza). Ver tabla siguiente para el resumen por servicio.

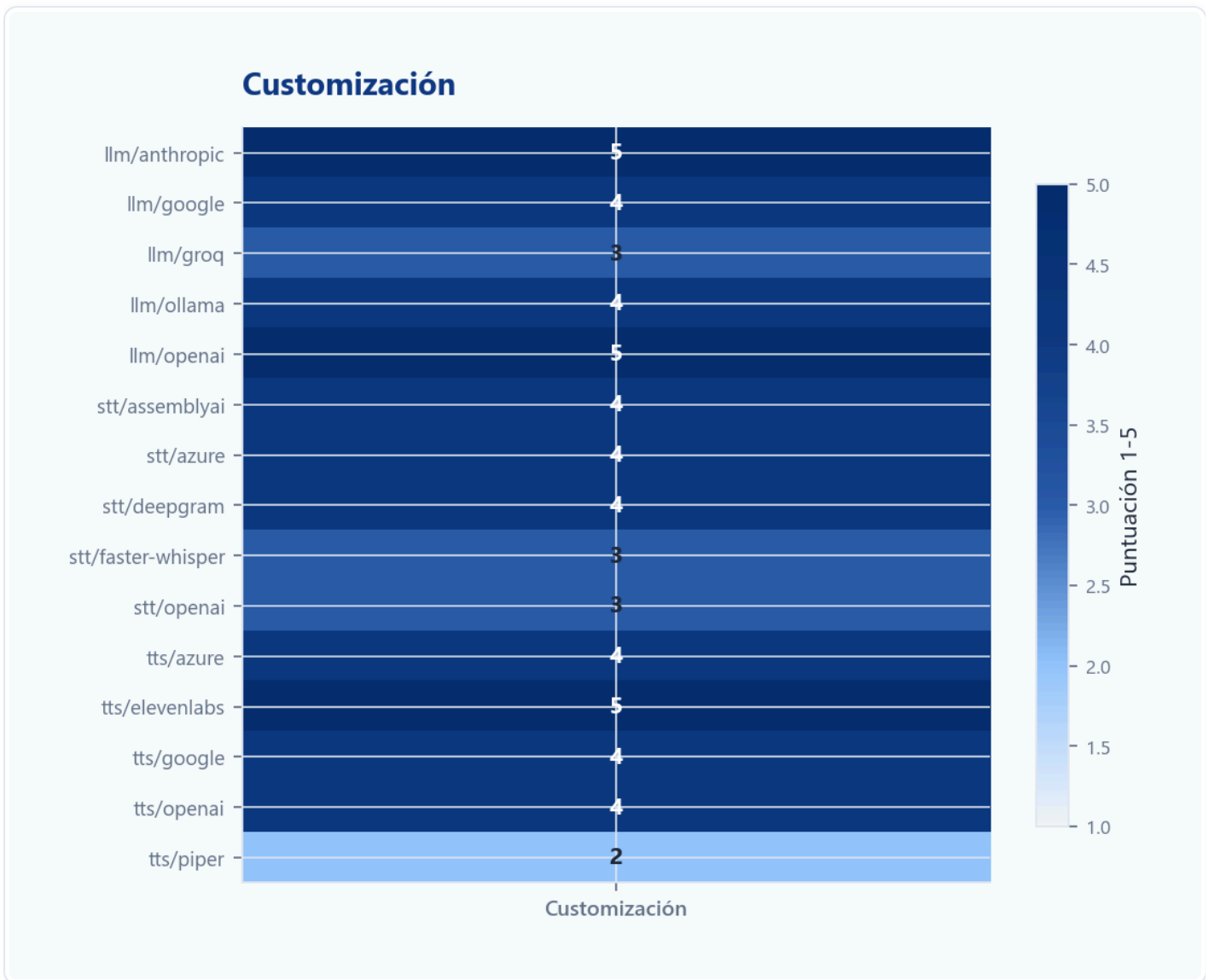
servicio	score	resumen
llm/anthropic	4	Nube, con política explícita de no usar prompts para entrenar; mejor que un free tier genérico, pero el dato igual viaja.
llm/google	3	Gemini pasa por Google Cloud; en plan de pago hay más control que en el tier gratis (revisar consola del proyecto).
llm/groq	4	Nube acelerada; declaran no entrenar con las llamadas. Para datos delicados haría falta contrato, no solo API key.
llm/ollama	5	Todo en el laptop; el prompt no sale a internet.
llm/openai	3	Texto y audio salen del equipo hacia OpenAI; la política API dice que no entrenan por defecto, pero sigue siendo nube.
stt/assemblyai	3	Nube; retención según el plan que contrates.

servicio	score	resumen
stt/azure	3	Datos en la región Azure que elijas; orientado a clientes con requisitos de cumplimiento.
stt/deepgram	3	Audio en nube; en contratos grandes se puede negociar retención mínima.
stt/faster-whisper	5	El audio no sale del equipo.
stt/openai	3	Subo el WAV a OpenAI; misma política de datos que el resto de la API.
tts/azure	3	Síntesis en la región Azure elegida.
tts/elevenlabs	3	Nube; clonar voz implica datos biométricos y consentimiento.
tts/google	3	Texto procesado en infra de Google; hace falta proyecto GCP.
tts/openai	3	El texto va a OpenAI para sintetizar; misma lógica de privacidad que el LLM.
tts/piper	5	Offline; el texto no se manda a terceros.

Mejores puntuaciones: - `llm/ollama` (5/5): Todo en el laptop; el prompt no sale a internet. - `stt/faster-whisper` (5/5): El audio no sale del equipo. - `tts/piper` (5/5): Offline; el texto no se manda a terceros.

Solo Ollama + faster-whisper + Piper evitan enviar voz o texto a internet; el resto son servicios en nube con notas 3-4 según retención.

Dimensión 5: Customización



Dimensión 5: Customización

Figura: mapa de calor (Customización). Ver tabla siguiente para el resumen por servicio.

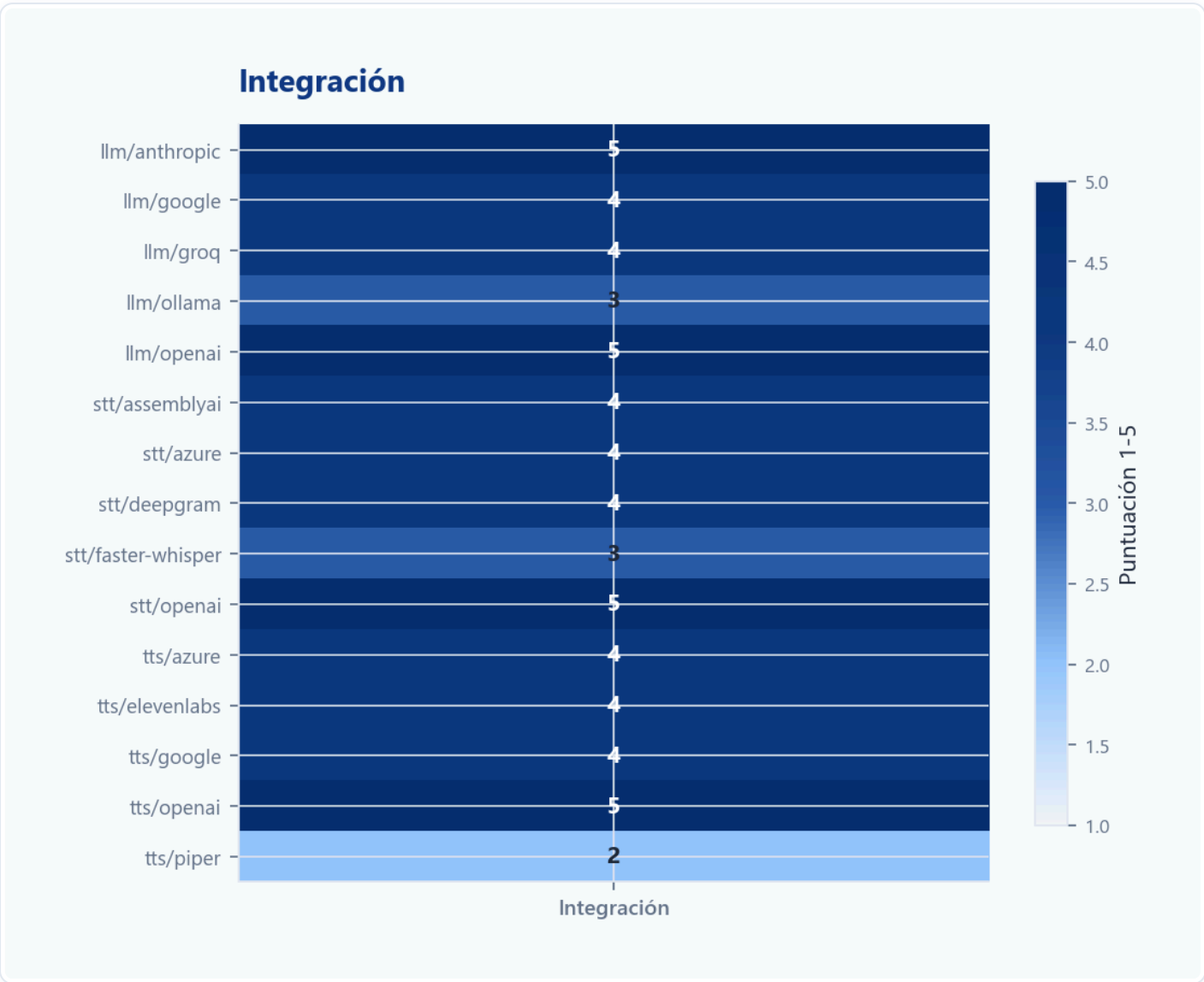
servicio	score	resumen
llm/anthropic	5	Buen margen en instrucciones largas, tools y cambiar entre modelos Claude.
llm/google	4	Instrucciones, filtros y parámetros sí; subir un modelo propio por API, no.
llm/groq	3	Pocos modelos publicados (Llama, Mixtral) y parámetros básicos; sin tools finas ni entrenar uno propio.
llm/ollama	4	Se cambia de modelo (Llama, Mistral), Modelfile y parámetros locales; no hay fine-tuning en nube.
llm/openai	5	El más flexible del grupo en LLM: system prompt, temperatura, varios GPT y modo JSON.
stt/assemblyai	4	Vocabulario custom, speaker labels y extras (resúmenes) en la misma API.

servicio	score	resumen
stt/azure	4	Modelos custom, frases guía y pronunciación; hay locale es-CR.
stt/deepgram	4	Varios Nova, idiomas, puntuación y diarización si pagás plan alto.
stt/faster-whisper	3	Elegís tamaño de modelo y GPU/CPU; no hay portal para entrenar con tus datos.
stt/openai	3	Idioma y un prompt de contexto; Whisper no se adapta fácil con datos propios.
tts/azure	4	Voces neurales, SSML y voz custom en planes altos.
tts/elevenlabs	5	Lo más flexible en voz: clonación, estilos y control expresivo (5/5 en escucha, Sección 3.0.5).
tts/google	4	Catálogo grande Neural2/Wavenet, SSML, velocidad y tono.
tts/openai	4	Varias voces y estilo en los modelos de voz recientes.
tts/piper	2	Solo voces ONNX pregrabadas; sin clonación ni matices de emoción.

Mejores puntuaciones: - [llm/anthropic](#) (5/5): Buen margen en instrucciones largas, tools y cambiar entre modelos Claude. - [llm/openai](#) (5/5): El más flexible del grupo en LLM: system prompt, temperatura, varios GPT y modo JSON. - [tts/elevenlabs](#) (5/5): Lo más flexible en voz: clonación, estilos y control expresivo (5/5 en escucha, Sección 3.0.5).

OpenAI y Anthropic ofrecen más opciones en LLM; ElevenLabs destaca en voz; Groq y Piper tienen menos ajustes finos.

Dimensión 6: Facilidad de integración



Dimensión 6: Facilidad de integración

Figura: mapa de calor (Facilidad de integración). Ver tabla siguiente para el resumen por servicio.

servicio	score	resumen
llm/anthropic	5	Paquete <code>anthropic</code> en Python; el stream de mensajes anduvo sin sorpresas en las pruebas.
llm/google	4	SDK <code>google-genai</code> ; tiene sentido si el proyecto ya vive en GCP.
llm/groq	4	API casi compatible con OpenAI: cambié URL y clave y corrió; fue el LLM más rápido aquí.
llm/ollama	3	HTTP en localhost; hay que tener el daemon arriba y no hay SLA de nadie.
llm/openai	5	SDK de Python con streaming; en este repo la misma cuenta cubre LLM, STT y TTS.
stt/assemblyai	4	SDK Python y REST claros para mandar audio y leer la transcripción.

servicio	score	resumen
stt/azure	4	SDK en Python y C#; lo usé pensando en Unity/.NET.
stt/deepgram	4	SDK reciente y WebSocket para vivo; en esta corrida fue el STT en nube más rápido.
stt/faster-whisper	3	Librería Python + CUDA; en el Quadro P2000 la latencia fue alta.
stt/openai	5	Una API key con LLM y TTS; pocas líneas de código para transcribir.
tts/azure	4	Misma clave que Azure STT; SDK o REST.
tts/elevenlabs	4	REST/SDK orientados a producto de voz; aquí fue el TTS más caro por carácter.
tts/google	4	Cliente oficial; hay que activar API y credenciales en la consola.
tts/openai	5	Misma API key que LLM y STT; devuelve audio directo.
tts/piper	2	Ejecutable o subprocess; no hay SDK cloud listo.

Mejores puntuaciones: - `llm/anthropic` (5/5): Paquete `anthropic` en Python; el stream de mensajes anduvo sin sorpresas en las pruebas. - `llm/openai` (5/5): SDK de Python con streaming; en este repo la misma cuenta cubre LLM, STT y TTS. - `stt/openai` (5/5): Una API key con LLM y TTS; pocas líneas de código para transcribir.

OpenAI unifica las tres capas con un solo SDK; Azure Speech va bien en .NET/Unity; los locales piden un proceso auxiliar o subprocess.

Selección por escenario (cualitativo)

escenario	llm	stt	tts	nota
Privacidad offline	llm/ollama (5)	stt/faster-whisper (5)	tts/piper (5)	Voz y texto no salen a internet; todo corre en el equipo.
Más opciones de voz y LLM	llm/openai o llm/anthropic (5)	stt/assemblyai (4)	tts/elevenlabs (5)	Más opciones de voz y herramientas; el costo en nube sube.
Integración rápida (un proveedor)	llm/openai (5)	stt/openai (5)	tts/openai (5)	Una API key, tres endpoints, SDK único.
Stack Microsoft	llm/openai (5) u otro	stt/azure (4)	tts/azure (4)	SDK Azure Speech, locale es-CR, útil si el cliente ya es .NET/Unity.

La tabla resume combinaciones habituales; latencia y costo numéricos están en Sección 3.0.6 y Sección 3.0.8.

4. Arquitectura y Pipeline de Comunicación

Pipeline de comunicación: Agente virtual (PF-3312)



15 servicios evaluados en STT, LLM y TTS · Ver docs/pipeline.puml para detalle UML

Pipeline de comunicación

Figura: flujo del agente por voz (STT → LLM → TTS). Los tiempos medidos en cadena están en Sección 3.0.8.

Diagrama UML detallado: [docs/pipeline.puml](#) .

Flujo: micrófono del cliente → STT → LLM (streaming) → TTS → reproducción de audio.

Los tiempos **medidos en cadena** (no solo sumados) aparecen en Pipeline end-to-end (Sección 3.0.8).

Integración en cliente (C# / Unity). En un agente 3D el mismo flujo se implementa desde el *game loop*: `Microphone` o `AudioClip` captura el audio del usuario; un `UnityWebRequest` o el SDK de Azure Speech envía el buffer a STT; la transcripción entra al LLM vía REST o streaming SSE; la respuesta textual se pasa a TTS y el `AudioSource` reproduce el clip devuelto. Ese es el punto de empalme con el agente del Proyecto 1 de este laboratorio: el audio sintetizado se reproduce en el `AudioSource` del avatar y puede alimentar el lip-sync por visemas ya configurado en esos modelos, de modo que la voz del agente sale “por la boca” del personaje y no como audio suelto. Azure Speech tiene SDK oficial para .NET y encaja con el escenario stack Azure de Costo por turno conversacional (Sección 3.0.6); OpenAI unifica las tres capas con una sola API key si se prioriza velocidad de desarrollo. Los componentes locales (Piper, faster-whisper, Ollama) se invocan por subprocess o HTTP local desde C# con `System.Diagnostics.Process` o un servicio sidecar en Python.

El repositorio incluye un **ejemplo conceptual en C#** para Unity en `examples/UnityPipelineClient.cs` (captura de micrófono, llamadas REST/SSE y reproducción con `AudioSource`), alineado con la implementación Python en `benchmarks/pipeline/` .

Tabla de integración: protocolos y SDK

Resumen documental para evaluar viabilidad en Python (benchmark) y C#/Unity (cliente del agente). Consulta mayo 2026.

servicio	sdk_python	sdk_csharp	rest	websocket_sse	grpc
llm/openai	Sí (oficial)	Parcial (REST/HttpClient)	Sí	SSE streaming	No
llm/anthropic	Sí (oficial)	REST manual	Sí	Stream de mensajes	No
llm/google	Sí (google-genai)	Sí (Google.Cloud.AIPlatform)	Sí	Streaming	Opcional vía GCP
llm/groq	Sí	REST manual	Sí (OpenAI-compatible)	SSE	No
llm/ollama	HTTP (requests)	REST local	Sí (localhost)	No	No
stt/deepgram	Sí (v6)	Sí	Sí	WebSocket tiempo real	No
stt/openai	Sí	REST manual	Sí	No (batch)	No
stt/assemblyai	Sí	REST manual	Sí	Streaming	No
stt/azure	Sí	Sí (Speech SDK.NET)	Sí	WebSocket	No
stt/faster-whisper	Librería local	Subprocess/sidecar	No cloud	No	No
tts/openai	Sí	REST manual	Sí	No	No
tts/elevenlabs	Sí	REST manual	Sí	Stream audio	No
tts/azure	Sí	Sí (Speech SDK)	Sí	WebSocket	No
tts/google	Sí	Sí (Google.Cloud.TextToSpeech)	Sí	No	Sí (cliente gRPC GCP)
tts/piper	CLI/subprocess	Process.Start	No	No	No

En Unity prioricé Azure Speech y Google TTS por SDK.NET; OpenAI, Groq y ElevenLabs van por REST desde C#. Para STT en vivo, Deepgram por WebSocket fue el más rápido aquí. Los locales piden un sidecar en Python o lanzar un proceso desde C#.

5. Recomendaciones y Conclusión

En la corrida del manifiesto se midieron los **15/15** servicios previstos. Con esos datos, estas son las combinaciones que mejor encajan con cada objetivo:

Escenario	Pipeline	Por qué
Tiempo real de baja latencia en nube	Groq + Deepgram + Google TTS	Turno E2E medido ≈ 1.8 s, $\sim \$0.01/\text{turno}$ (Pipeline end-to-end, Sección 3.0.8)
Calidad premium	OpenAI o Anthropic + AssemblyAI + ElevenLabs	Mejor razonamiento y voz al oído; E2E ≈ 9.0 s
Privacidad estricta y presupuesto cero	Ollama + faster-whisper + Piper	100 % offline, \$0/llamada; E2E ≈ 62 s en este hardware (Sección 2.5 y 3.0.8)
Cliente.NET / Unity	Azure STT + TTS (es-CR) + LLM a elección	SDK oficial de Azure Speech en.NET/Unity, voz costarricense; E2E ≈ 7.8 s

Cada fila corresponde a un escenario del Pipeline end-to-end (Sección 3.0.8): Demo rápida nube, Calidad premium, Privacidad offline y Stack Azure, respectivamente.

Conclusión. Cada capa destaca en algo distinto. Groq y Deepgram sirven para tiempo real; OpenAI y ElevenLabs para calidad percibida; el stack local maximiza privacidad pero no velocidad en este hardware (~ 62 s E2E de promedio). Hay que medir el pipeline completo (Sección 3.0.8), no solo sumar latencias aisladas: el tiempo entre etapas y el TTFT del LLM cambian lo que siente el usuario. Si cambian la GPU, la red o la versión de la API, repetiría la corrida antes de decidir stack.

Trabajo futuro. Se podría medir streaming bidireccional (WebSocket o gRPC), donde el usuario escucha la primera sílaba antes de terminar el turno; probar varios turnos a la vez; medir consumo eléctrico con wattímetro para comparar local vs nube; grabar más audios con acento costarricense en condiciones reales (el WER con FLEURS fue menor que con Piper sintético); y repetir el benchmark en una GPU más nueva para ver si el stack local se acerca al tiempo real.

6. Referencias

Tarifas oficiales de los proveedores (consultadas el 8 de junio de 2026; los precios usados en **Tarifas en unidades estándar** y en las tablas de costo del informe corresponden a estas páginas):

- OpenAI. *API Pricing*. <https://openai.com/api/pricing/>
- Anthropic. *Claude API Pricing*. <https://www.anthropic.com/pricing>
- Google. *Gemini API Pricing*. <https://ai.google.dev/pricing>
- Groq. *Pricing*. <https://groq.com/pricing>
- Deepgram. *Pricing*. <https://deepgram.com/pricing>
- AssemblyAI. *Pricing*. <https://www.assemblyai.com/pricing>
- Microsoft Azure. *AI Speech Pricing*. <https://azure.microsoft.com/pricing/details/cognitive-services/speech-services/>
- Google Cloud. *Text-to-Speech Pricing*. <https://cloud.google.com/text-to-speech/pricing>
- ElevenLabs. *Pricing*. <https://elevenlabs.io/pricing>

Datasets y herramientas:

- Conneau, A. et al. (2022). *FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech*. arXiv:2205.12446. Dataset: <https://huggingface.co/datasets/google/fleurs>
- Ardila, R. et al. (2020). *Common Voice: A Massively-Multilingual Speech Corpus*. arXiv:1912.06670. Proyecto: <https://commonvoice.mozilla.org/>
- Radford, A. et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. arXiv:2212.04356.
- Ollama (inferencia LLM local). <https://ollama.com>
- SYSTRAN. *faster-whisper* (STT local sobre CTranslate2). <https://github.com/SYSTRAN/faster-whisper>
- Rhasspy. *Piper TTS* (síntesis de voz local). <https://github.com/rhasspy/piper>
- *jiwer* (cálculo de Word Error Rate). <https://github.com/jitsi/jiwer>

Integración con agente virtual (Proyecto 1):

- Unity Technologies. *AudioSource* (reproducción de audio en cliente 3D). <https://docs.unity3d.com/Manual/class-AudioSource.html>
- Microsoft. *Azure Speech SDK for.NET*. <https://learn.microsoft.com/azure/ai-services/speech-service/quickstarts/setup-platform>

Apéndice A: Archivos del repositorio

Carpeta / archivo	Contenido
results/	CSV por categoría, E2E, manifiesto, hardware, speedtest
docs/graficos_generados/	Gráficos de latencia, costo, heatmaps, pipeline
docs/dimensiones_datos/	Matrices y tablas por dimensión (derivadas de los CSV)
data/	Prompts, transcripciones de referencia, puntuaciones cualitativas
benchmarks/	Scripts de medición por proveedor y pipeline E2E
examples/UnityPipelineClient.cs	Ejemplo conceptual del pipeline en cliente Unity
Repositorio GitHub	Código y resultados en <a href="https://github.com/<tu-usuario>/lab_entregable_2">https://github.com/<tu-usuario>/lab_entregable_2

Apéndice B: Inputs controlados de prueba

Reproducción de los insumos definidos en `common/prompts.py`, `common/audio_samples.py` y `data/reference_transcriptions.json`.

B.1 Prompts LLM (5)

p1_factual_corto : Pregunta factual corta

p2_razonamiento : Razonamiento multi-paso

Un sistema de agentes virtuales ofrece tres avatares 3D: Alfa (28 mil triángulos), Beta (31 mil triángulos) y Gamma (17 mil triángulos). Si la GPU del cliente soporta como máximo 50 mil triángulos simultáneos en pantalla, indique paso a paso qué combinaciones se pueden mostrar a la vez sin exceder el límite, y cuál combinación ofrece la mayor variedad visual sin pasarse del presupuesto.

p3_json_estriicto : Instrucciones con formato JSON estricto

Devuelva EXCLUSIVAMENTE un objeto JSON válido, sin texto adicional, que liste tres agentes virtuales con los campos: nombre (string), estilo (uno de: realista, caricaturesco, cartoon), rol (string), expresiones (array de strings). No incluya comentarios ni markdown.

p4_contexto_largo : Resumen de contexto largo

A continuación encontrará un fragmento técnico. Resúmalo en máximo 60 palabras, conservando los tres puntos principales:

Texto: En el diseño del tutor de voz del laboratorio se plantearon tres metas: que la respuesta en nube llegue en menos de dos segundos, que no se guarde audio sin un consentimiento explícito del usuario, y que se pueda cambiar la voz del avatar sin recompilar la aplicación Unity. También se propuso estimar el costo por clase de quince minutos y revisar el error de transcripción con grabaciones de sala, no solo con audios sintéticos de Piper.

p5_creativo : Generación creativa / role-play

Usted es un coach virtual de bienestar físico. Salude al usuario en máximo cuatro oraciones, ofrezca una rutina corta de calentamiento de tres ejercicios, y termine con una frase motivadora. Use un tono cercano pero profesional.

B.2 Textos TTS (5)

- **t1_saludo_corto** : Hola, ¿cómo estás? Soy un agente virtual de prueba.
- **t2_oracion_media** : Hola. En esta prueba cada proveedor leerá la misma frase para comparar claridad y ritmo de la voz sintetizada.
- **t3_parrafo_largo** : El tutor virtual permite elegir voz masculina o femenina, activar subtítulos en pantalla y descargar un resumen de la sesión. Si el usuario cambia el idioma, la conversación reinicia desde el saludo inicial. También puede silenciar la síntesis y leer solo texto.
- **t4_terminos_tecnicos** : El agente virtual procesa audio a dieciséis kilohertz, calcula latencia al primer token y sintetiza respuestas de voz en streaming.
- **t5_dialogo_cr** : Mae, ¿podés explicarme en pocas palabras cómo elegir entre Groq, OpenAI y un modelo local para el tutor virtual?

B.3 Audios STT (14)

- **a1_saludo_corto** (synthetic_piper): «Hola, buenos días. Soy el asistente de voz del tutor del curso.»
- **a2_oracion_media** (synthetic_piper): «Hoy probamos cómo cada motor de voz entiende una frase media con el micrófono a un metro de distancia.»
- **a4_parrafo_tecnico** (synthetic_piper): «El sistema graba a dieciséis kilohertz, manda el audio al servicio de transcripción y muestra el texto en pantalla antes de pasarlo al modelo de lenguaje.»
- **a5_acentos_cr** (synthetic_piper): «Mae, qué tuanis lo que hizo el agente virtual hoy. La verdad sí jaló pura vida con la transcripción en tiempo real.»
- **g1_fleurs** (fleurs_human): «los murales o garabatos indeseados reciben el nombre de grafiti»
- **g2_fleurs** (fleurs_human): «por el momento no se sabe qué cargos se imputarán o qué condujo a las autoridades hasta el niño pero se ha dado inicio a procedimientos penales de menores en el tribunal federal»
- **g4_fleurs** (fleurs_human): «a veces un mismo vuelo puede costar precios muy diferentes en varios recopiladores de contenidos por lo que vale la pena comparar los resultados de búsqueda y navegar en el sitio web de la empresa antes de realizar la reserva»
- **c1_cv_noisy** (common_voice_noisy): «Por un lado, en el sector culto encontramos un movimiento progresivamente creciente.»
- **c3_cv_noisy** (common_voice_noisy): «Generalmente se celebra el tercer sábado del mes de septiembre.»
- **r1_cv_centroamerica** (common_voice_central_america): «Situado en la provincia de Guadalajara, en la comarca del Señorío de Molina-Alto Tajo.»
- **r2_cv_centroamerica_largo** (common_voice_central_america): «Situado en la provincia de Guadalajara, en la comarca del Señorío de Molina-Alto Tajo. Sin embargo, la investigación paleontológica para buscar conexiones evolutivas tiene ciertas limitaciones. La Real Academia Española define orientador como "que orienta".»
- **l1_fleurs_largo** (fleurs_human_concat): «por el momento no se sabe qué cargos se imputarán o qué condujo a las autoridades hasta el niño pero se ha dado inicio a procedimientos penales de menores en el tribunal federal a veces un mismo vuelo puede costar precios muy diferentes en varios recopiladores de contenidos por lo que vale la pena comparar los resultados de búsqueda y navegar en el sitio web de la empresa antes de realizar la reserva»
- **l2_piper_largo** (synthetic_piper_long): «El tutor de voz del laboratorio escucha al estudiante por el micrófono del laptop, muestra en pantalla lo que entendió y responde con audio generado. Si la red falla, la aplicación puede cambiar a un modo local que tarda más pero no envía datos afuera. En las pruebas del curso se midió el tiempo de cada paso por separado y también el turno completo, porque sumar latencias aisladas no siempre coincide con lo que siente el usuario en la sala. Por eso en el protocolo de benchmark también se registró el costo por clase de quince minutos y se comparó el error de transcripción entre grabaciones de sala y archivos sintéticos.»

- **l4_noisy_largo** (synthetic_noisy_long): «por el momento no se sabe qué cargos se imputarán o qué condujo a las autoridades hasta el niño pero se ha dado inicio a procedimientos penales de menores en el tribunal federal a veces un mismo vuelo puede costar precios muy diferentes en varios recopiladores de contenidos por lo que vale la pena comparar los resultados de búsqueda y navegar en el sitio web de la empresa antes de realizar la reserva»